

STAC32 Assignment 2

Due Thursday September 20 at 11:59pm

As before, any questions without solutions (the last two, here) are an assignment: you need to do these questions yourself and hand them in (instructions below). The remaining questions have my solutions attached; these are for you to work through, eg. in tutorial. They may contain hints for the questions to hand in.

The assignment is due on the date shown above. An assignment handed in after the deadline is late, and may or may not be accepted (see course outline). My solutions to the assignment questions will be available when everyone has handed in their assignment.

You are reminded that work handed in with your name on it must be *entirely your own work*.

Assignments are to be handed in on Quercus. See <https://www.utsc.utoronto.ca/~butler/c32/quercus1.nb.html> for instructions on handing in assignments in Quercus. Markers' comments and grades will be available there as well.

You will probably want to begin with this:

```
library(tidyverse)
```

(If you have opened a new project on `rstudio.cloud` for this assignment, you will most likely need `install.packages("tidyverse")` first.)

On we go.

1. The data in file <http://www.utsc.utoronto.ca/~butler/c32/ncbirths.csv> are about 500 randomly chosen births of babies in North Carolina. There is a lot of information: not just the weight at birth of the baby, but whether the baby was born prematurely, the ages of the parents, whether the parents are married, how long (in weeks) the pregnancy lasted (this is called the “gestation”) and so on.
 - (a) Read in the data from the file into R, bearing in mind what type of file it is.

Solution:

This is a `.csv` file (it came from a spreadsheet), so it needs reading in accordingly. Work directly from the URL (rather than downloading the file, unless you are working offline):

```
myurl="http://www.uts.utoronto.ca/~butler/c32/ncbirths.csv"
bw=read_csv(myurl)

## Parsed with column specification:
## cols(
##   'Father Age' = col_integer(),
##   'Mother Age' = col_integer(),
##   'Weeks Gestation' = col_integer(),
##   'Pre-natal Visits' = col_integer(),
##   'Marital Status' = col_integer(),
##   'Mother Weight Gained' = col_integer(),
##   'Low Birthweight?' = col_integer(),
##   'Weight (pounds)' = col_double(),
##   'Premie?' = col_integer(),
##   'Few Visits?' = col_integer()
## )
```

This shows you which variables the data set has (some of the names got a bit mangled), and it shows you that they are all integers except for the birth weight (a decimal number).

The easiest way to find out how many rows and columns there are is simply to list the data frame:

```
bw
## # A tibble: 500 x 10
##   `Father Age` `Mother Age` `Weeks Gestatio~ `Pre-natal Visi~
##           <int>         <int>         <int>         <int>
## 1             27            26            38            14
## 2             35            33            40            11
## 3             34            22            37            10
## 4             NA            16            38             9
## 5             35            33            39            12
## 6             32            24            36            12
## 7             33            33            38            15
## 8             38            35            38            16
## 9             28            29            40             5
## 10            NA            19            34            10
## # ... with 490 more rows, and 6 more variables: `Marital Status` <int>,
## #   `Mother Weight Gained` <int>, `Low Birthweight?` <int>, `Weight
## #   (pounds)` <dbl>, `Premie?` <int>, `Few Visits?` <int>
```

or you can take a “glimpse” of it:

```
glimpse(bw)

## Observations: 500
## Variables: 10
## $ `Father Age`      <int> 27, 35, 34, NA, 35, 32, 33, 38, 28, NA,...
## $ `Mother Age`      <int> 26, 33, 22, 16, 33, 24, 33, 35, 29, 19,...
## $ `Weeks Gestation`  <int> 38, 40, 37, 38, 39, 36, 38, 38, 40, 34,...
## $ `Pre-natal Visits` <int> 14, 11, 10, 9, 12, 12, 15, 16, 5, 10, 1...
## $ `Marital Status`   <int> 1, 1, 2, 2, 1, 1, 2, 1, 1, 2, 1, 1, 2, ...
## $ `Mother Weight Gained` <int> 32, 23, 50, NA, 15, 12, 60, 2, 20, NA, ...
## $ `Low Birthweight?` <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, ...
## $ `Weight (pounds)`  <dbl> 6.8750, 6.8125, 7.2500, 8.8125, 8.8125,...
## $ `Premie?`          <int> 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, ...
## $ `Few Visits?`      <int> 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, ...
```

Either of these displays show that there are 500 rows (observations, here births) and 10 columns (variables), and they both show what the variables are called. So they're both good as an answer to the question.

What you'll notice is that the variables have *spaces* in their names, which will require special handling later. These outputs show you what to do about those spaces in variable names: surround the variable name with "backticks". (On my keyboard, that's on the key to the left of number 1, where the squiggle is, that looks like a backwards apostrophe. Probably next to **Esc**, depending on the layout of your keyboard.)

Although almost all of the variables are stored as integers, the ones that have a question mark in their name are actually "logical", true or false, with 1 denoting true and 0 false. We could convert them later if we want to. (A question mark is not a traditional character to put in a variable name, so we have to surround these variables with backticks too.)

- (b) From your output, verify that you have the right number of observations and that you have several variables. Which of your variables correspond to birthweight, prematurity and length of pregnancy? (You might have to make guesses based on the names of the variables.)

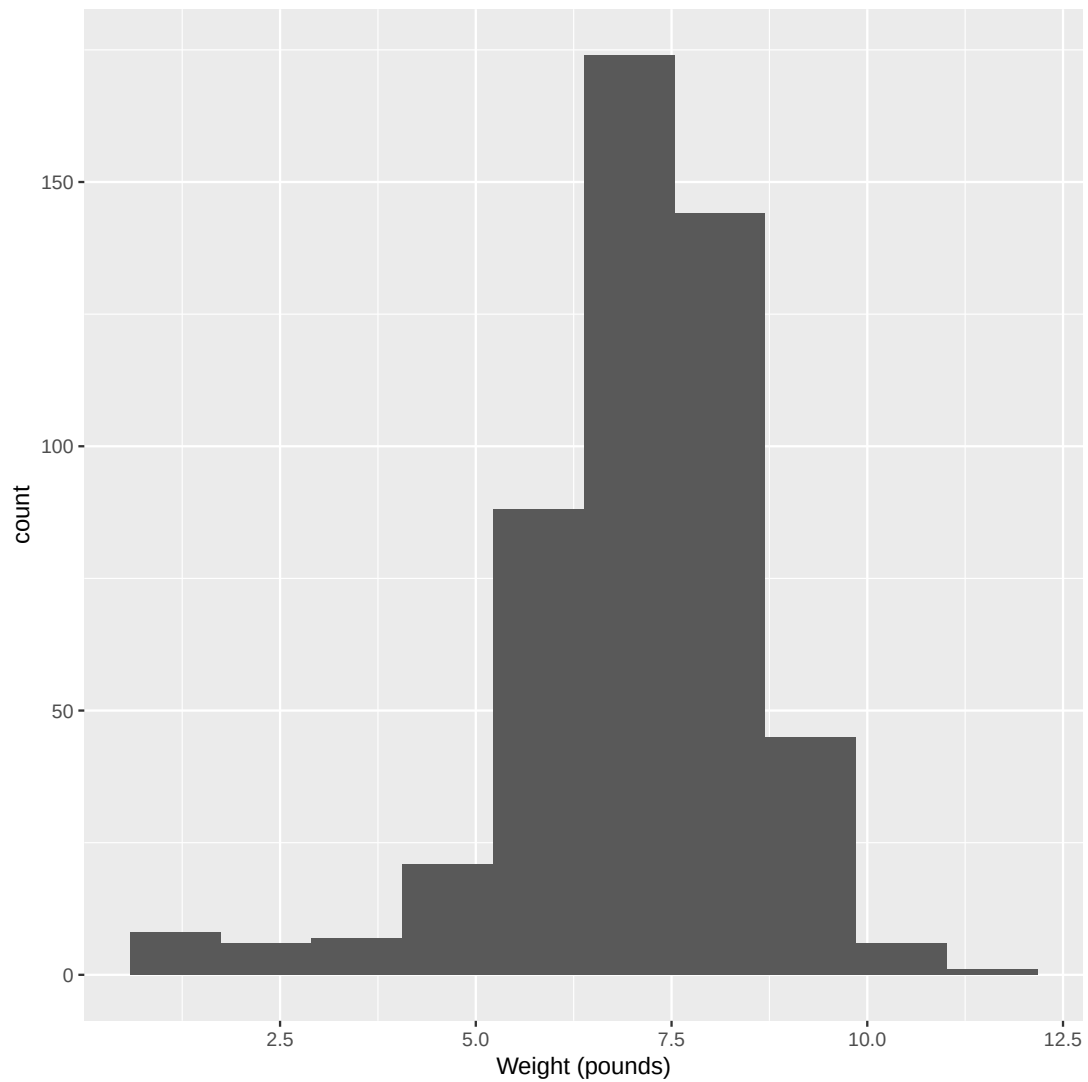
Solution: I do indeed have 500 observations on 10 variables ("several"). (If you don't have several variables, check to see that you didn't use `read_delim` or something by mistake.) After the "500 observations of 10 variables" line(s) in each case, you see all the variables by name, with what type of values they have¹, and the first few of the values.²

The variable ``Weight (pounds)`` is the birthweight (in pounds), ``Premie?`` is 1 for a premature baby and 0 for a full-term baby, and ``Weeks Gestation`` is the number of weeks the pregnancy lasted. I've put backticks around each of those to remind us that that's how we'll need to refer to them when we use them later.³

- (c) The theory behind the *t*-test (which we do later) says that the distribution of birth weights should be (approximately) normally distributed. Obtain a histogram of the birth weights. Does it look approximately normal? Comment briefly. (You'll have to pick a number of bins for your histogram first. I don't mind very much what you pick, as long as it's not obviously too many or too few bins.)

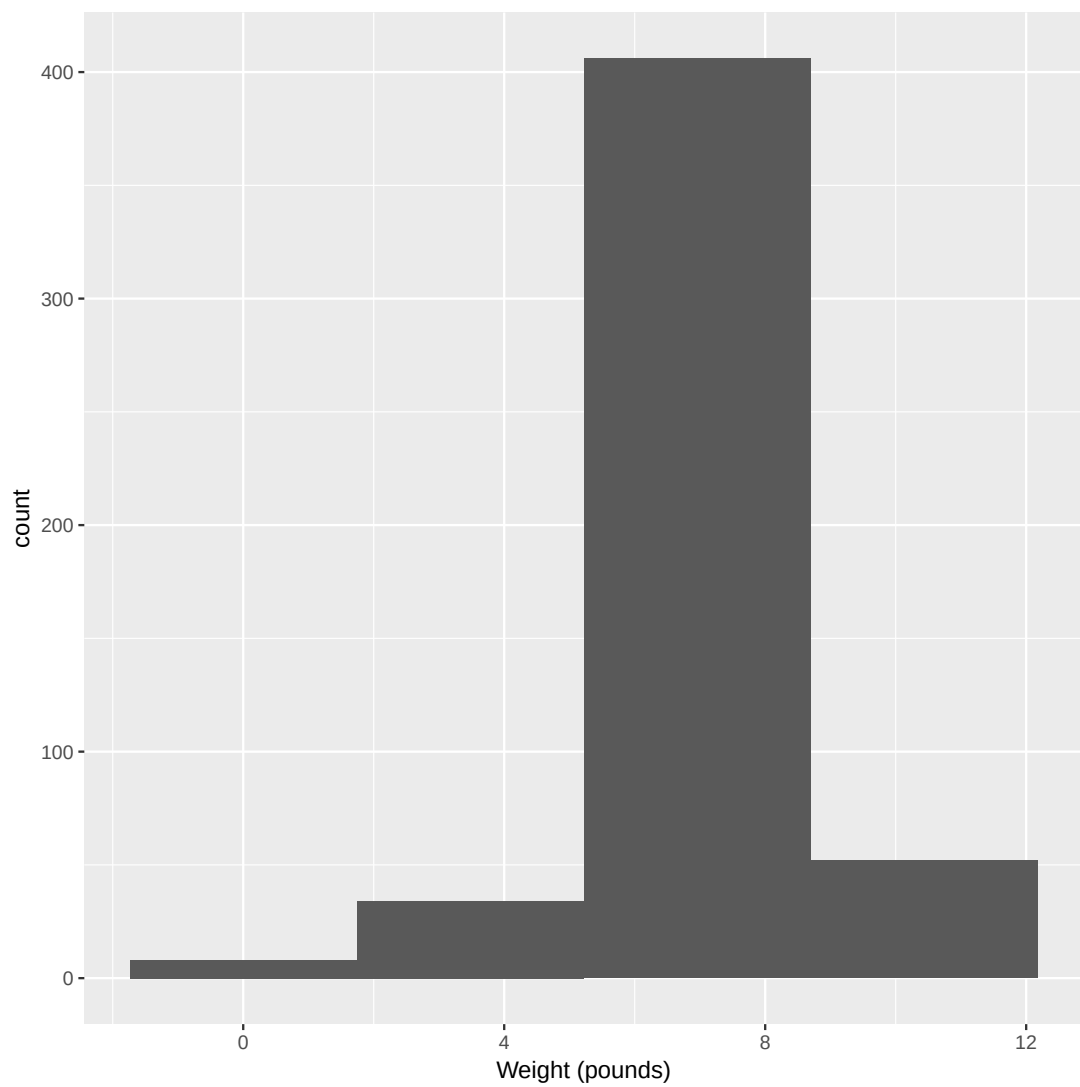
Solution: You'll have seen that I often start with 10 bins, or maybe not quite that many if I don't have much data, and this is a decent general principle. That would give

```
ggplot(bw,aes(x=`Weight (pounds)`))+geom_histogram(bins=10)
```



which is perfectly acceptable. You can try something a bit more or a bit less, and see how you like it in comparison. What you are looking for is a nice clear picture of *shape*. If you have too few bins, you'll lose the shape:

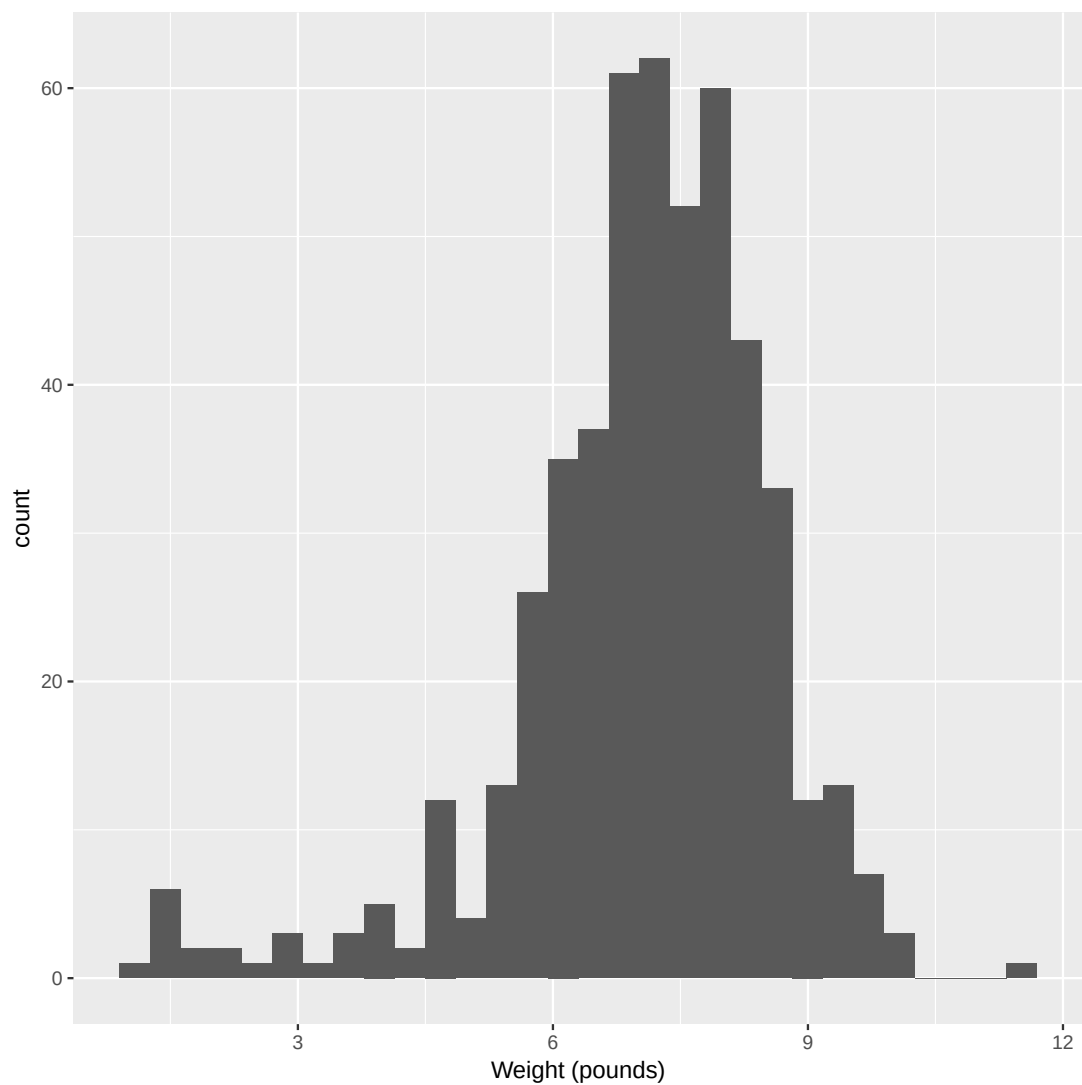
```
ggplot(bw,aes(x=`Weight (pounds)`))+geom_histogram(bins=4)
```



(is that leftmost bin an indication of skewness or some observations that happen to be smallish?)

And if you have too many, the shape will be there, but it will be hard to make out in all the noise, with frequencies going up and down:

```
ggplot(bw, aes(x=`Weight (pounds)`))+geom_histogram(bins=30)
```



I generally am fairly relaxed about the number of bins you use, as long as it's not clearly too few or too many. You might have done exercises in the past that illustrate that the choice of number of bins (or the class intervals where you move from one bin to the next, which is another issue that I won't explore here) can make an appreciable difference to how a histogram looks.

Extra: I had some thoughts about this issue that I put in a blog post, that you might like to read: <http://ritsokiguess.site/docs/2017/06/08/histograms-and-bins/>. The nice thing about Sturges' rule, mentioned there, is that you can almost get a number of bins for your histogram in your head (as long as you know the powers of 2, that is). What you do is to start with your sample size, here $n = 500$. You find the next power of 2 above that, which is here $512 = 2^9$. You then take that power and add 1, to get 10 bins. If you don't like that, you can get R to calculate it for you:

```
nclass.Sturges(bw$`Weight (pounds)`)
## [1] 10
```

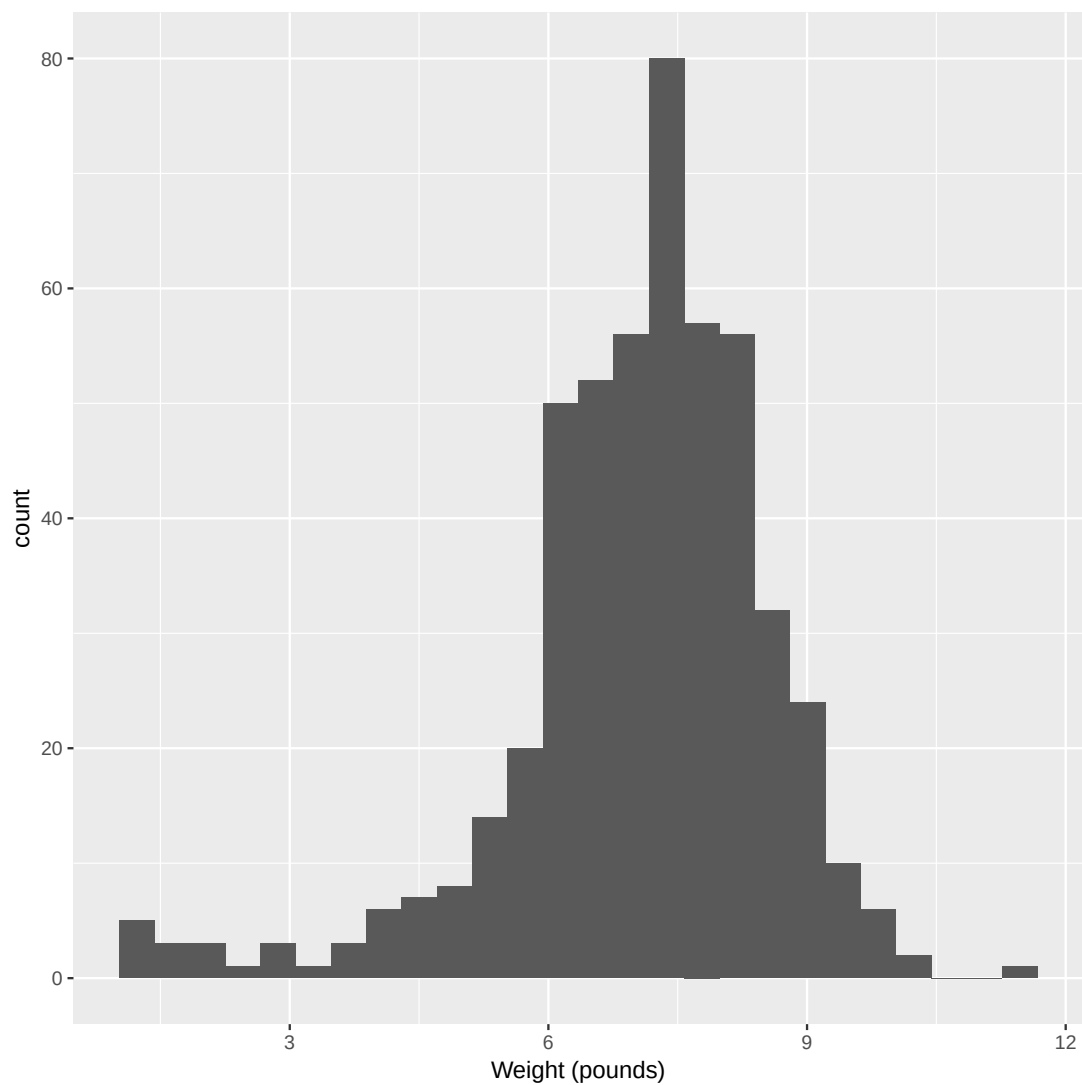
The place where Sturges' rule comes from is an assumption of normal data (actually a binomial approximation to the normal, backwards though that sounds). If you have less than 30 observations, you'll get fewer than 6 bins, which won't do much of a job of showing the shape. Rob Hyndman wrote a critical note about Sturges' rule in which he asserts that it is just plain wrong (if you have taken B57, this note is very readable).

So what to use instead? Well, judgment is still better than something automatic, but if you want a place to start from, something with a better foundation than Sturges is the Freedman-Diaconis rule. This, in its original formulation, gives a bin width rather than a number of bins:

$$w = 2(IQR)n^{-1/3}$$

The nice thing about this is that it uses the interquartile range, so it won't be distorted by outliers. `geom_histogram` can take a bin width, so we can use it as follows:

```
w=2*IQR(bw$`Weight (pounds)`)*500^(-1/3)
w
## [1] 0.4094743
ggplot(bw,aes(x=`Weight (pounds)`))+geom_histogram(binwidth=w)
```



R also has

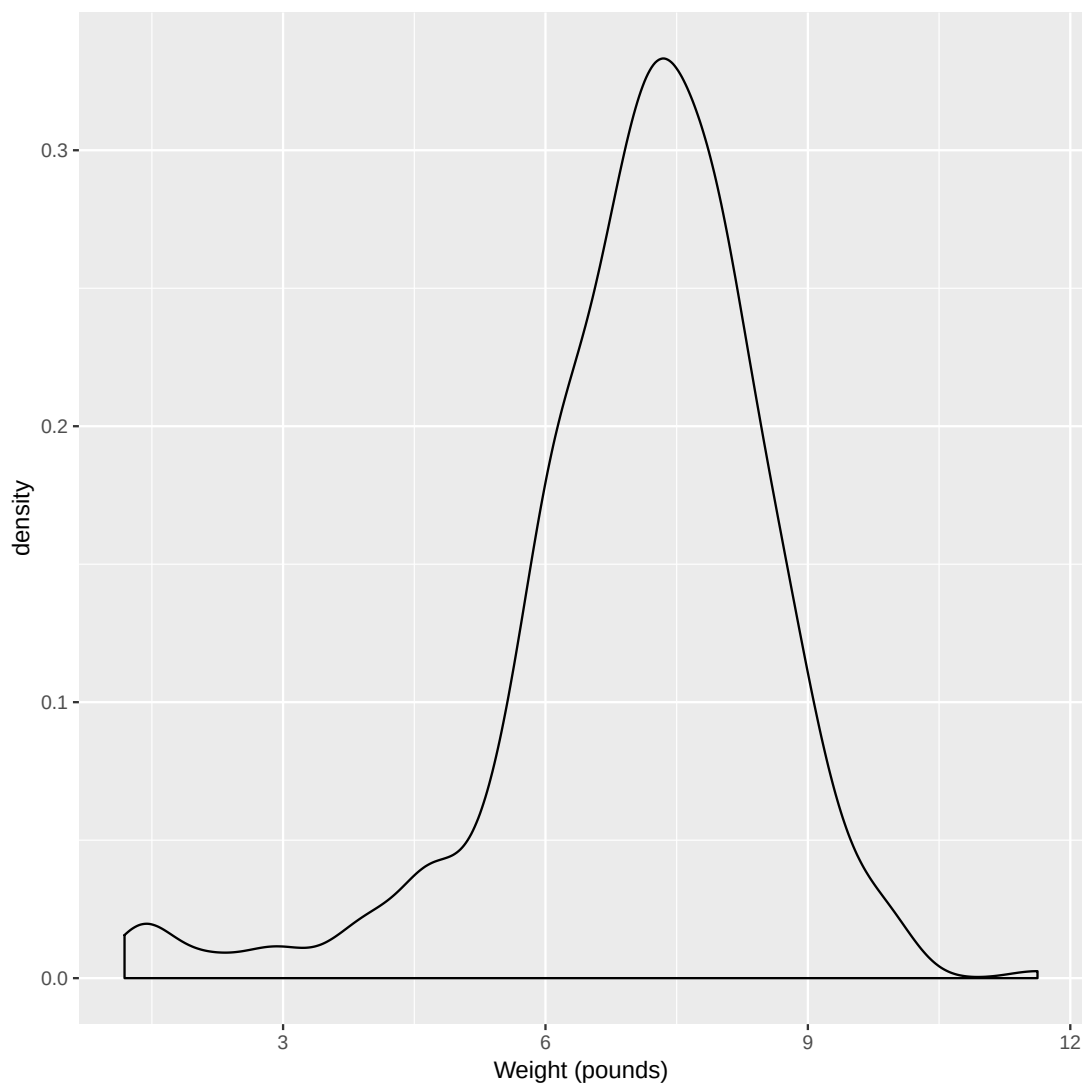
```
nc=nclass.FD(bw$`Weight (pounds)`)
nc
## [1] 26
```

which turns the Freedman-Diaconis rule into a number of bins rather than a binwidth; using that gives the same histogram as we got with `binwidth`.

In my opinion, Freedman-Diaconis tends to give too many bins (here there are 26 rather than the 10 of Sturges). But I put it out there for you to make your own call.

Another way to go is a “density plot”. This is a smoothed-out version of a histogram that is not obviously frequencies in bins, but which does have a theoretical basis. It goes something like this:

```
ggplot(bw, aes(x=`Weight (pounds)`))+geom_density()
```



`geom_density` has an optional parameter that controls how smooth or wiggly the picture is, but the default is usually good.

Alright, before we got distracted, we were assessing normality. What about that?

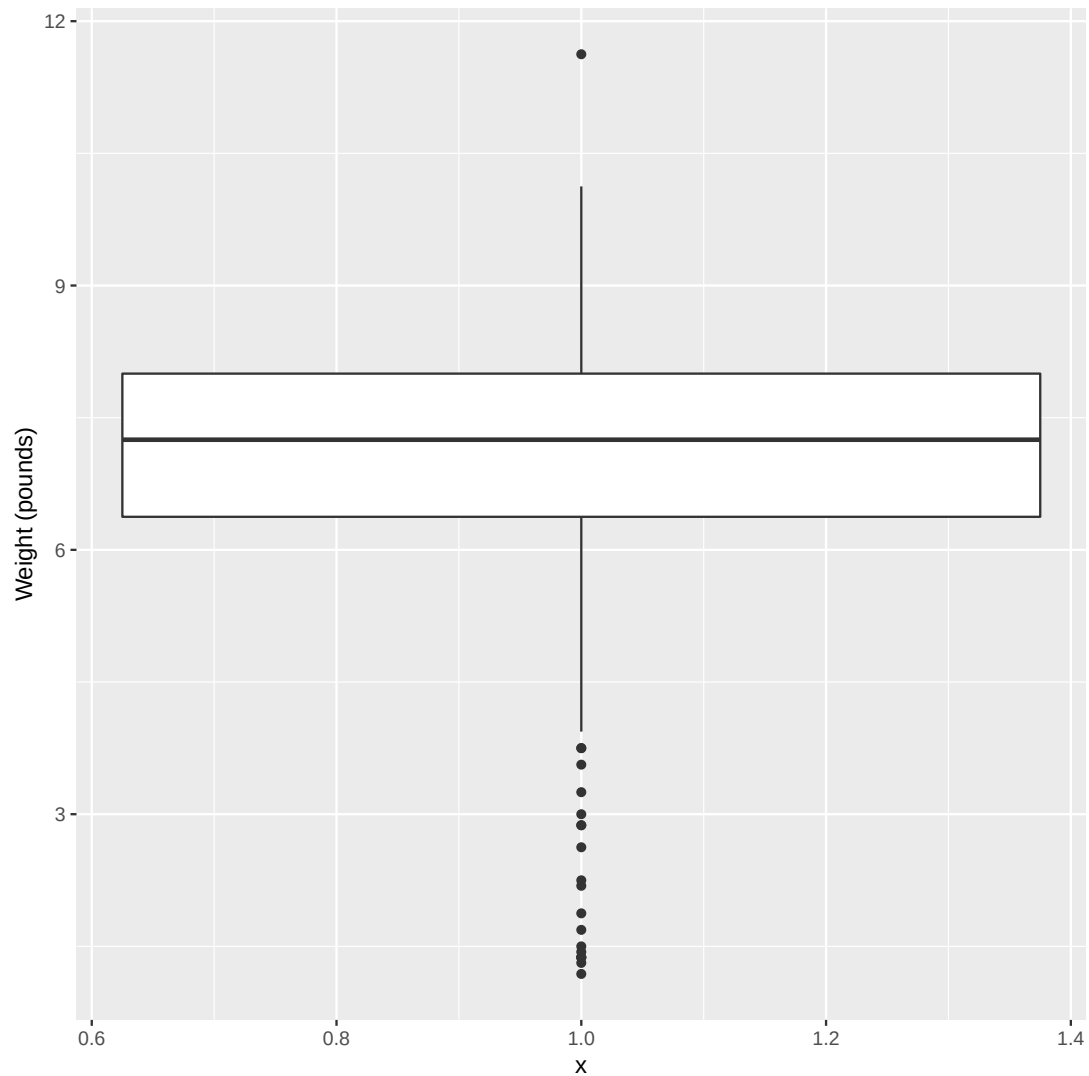
It is mostly normal-looking, but I am suspicious about those *very* low birth weights, the ones below about 4 pounds. There are a few too many of those, as I see it.

If you think this is approximately normal, you need to make some comment along the lines of “the shape is approximately symmetric with no outliers”. I think my first answer is better, but this answer is worth something, since it is a not completely unreasonable interpretation of the histogram.

I have been making the distinction between a histogram (for one quantitative variable) and side-by-side boxplots (for one quantitative variable divided into groups by one categorical variable). When you learned the boxplot, you probably learned it in the context of one quantitative variable. You can draw a boxplot for that, too, but the `ggplot` boxplot has an `x` as well as a `y`. What you do to make a single boxplot is to set the `x` equal 1, which produces a weird *x*-axis

(that you ignore):

```
ggplot(bw,aes(x=1,y=`Weight (pounds)`))+geom_boxplot()
```



The high weight is actually an outlier, but look at all those outliers at the bottom!⁴

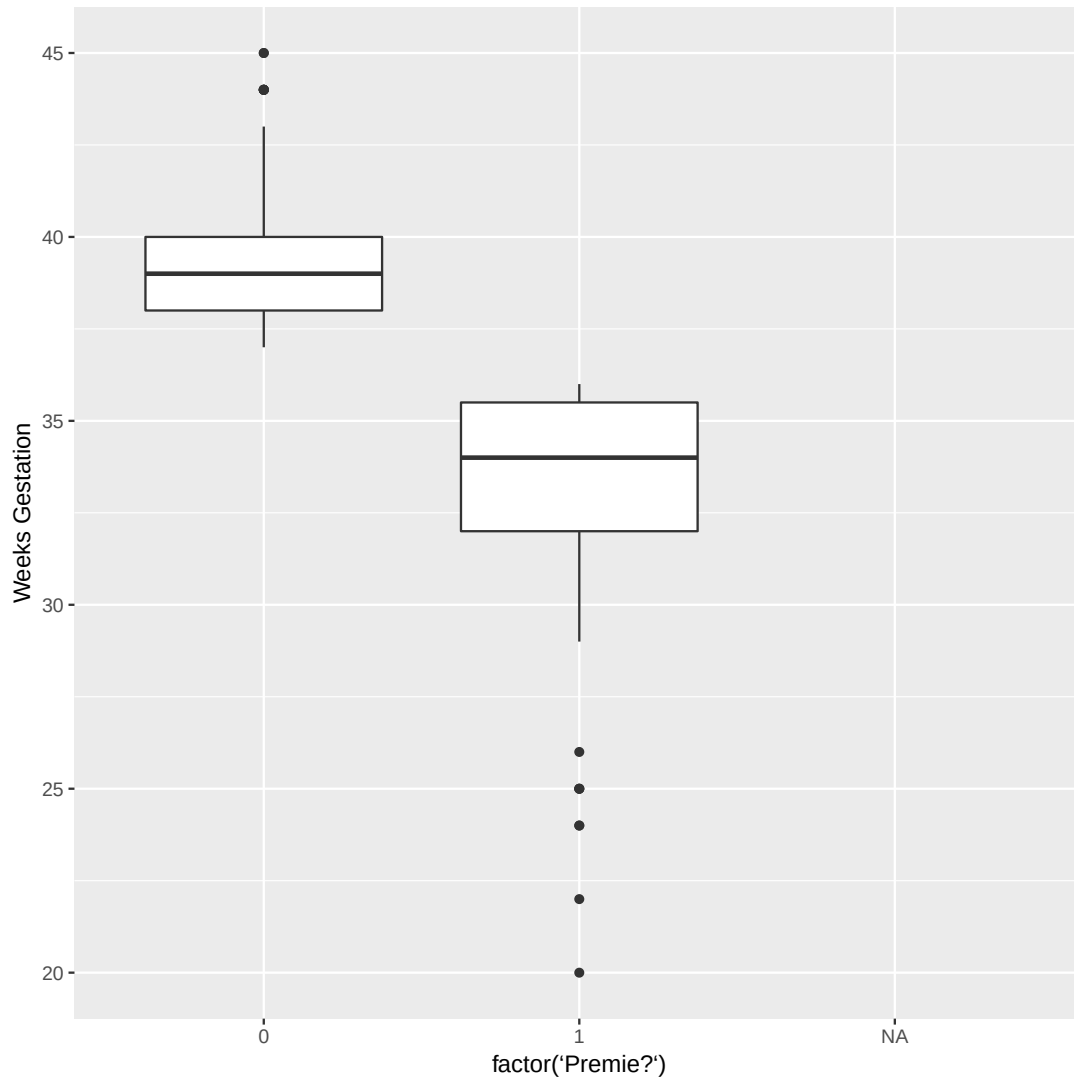
I think the reason for those extra very low values is that they are the premature births (that can result in *very* small babies). Which leads to the additional question coming up.

This is a continuation of the previous question:

2. This is an exploration of some extra issues around the North Carolina births data set.
 - (a) How short does a pregnancy have to be, for the birth to be classified as “premature”? Deduce this from the data, by drawing a suitable graph or otherwise.

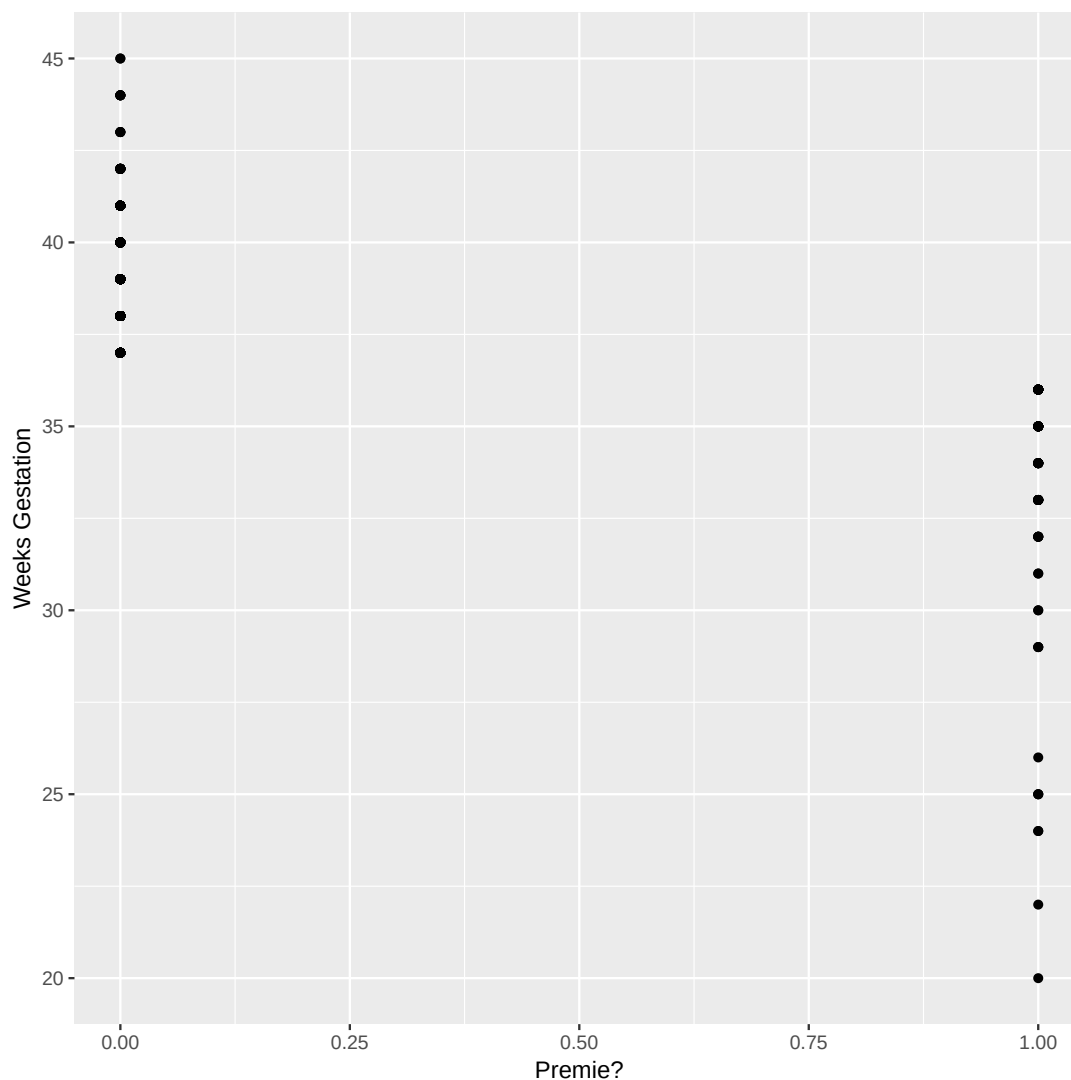
Solution: To figure it out from the data, we can see how 'Weeks Gestation' depends on 'Premie?'. Some possibilities are boxplots or a scatterplot. Either of the first two graphs would get full credit (for the graphing part: you still have to do the explanation) if this were being marked:

```
ggplot(bw,aes(x=factor(`Premie?`),y=`Weeks Gestation`))+geom_boxplot()  
## Warning: Removed 1 rows containing non-finite values (stat_boxplot).
```



The warning is because the prematurity of one of the babies is not known. Or

```
ggplot(bw,aes(x=`Premie?`,y=`Weeks Gestation`))+geom_point()  
## Warning: Removed 1 rows containing missing values (geom_point).
```



The same warning again, for the same reason.

Notice how the graphs are similar in syntax, because the what-to-plot is the same (apart from the **factor** thing) and we just make a small change in how-to-plot-it. In the boxplot, the thing on the *x*-scale needs to be categorical, and `Premie?` is actually a number, so we'd better make it into a **factor**, which is R's version of a categorical variable. `Premie.` is actually a categorical variable ("premature" or "not premature") masquerading as a quantitative one (1 or 0). It is an "indicator variable", if you're familiar with that term.

It looks as if the breakpoint is 37 weeks: a pregnancy at least that long is considered normal, but a shorter one ends with a premature birth. Both plots show the same thing: the `Premie?=1` births all go with short pregnancies, shorter than 37 weeks. This is completely clear cut.

Another way to attack this is to use `summarize`, finding the max and min:

```
bw %>% summarize( n=n(),
                  min=min(`Weeks Gestation`),
                  max=max(`Weeks Gestation`))

## # A tibble: 1 x 3
##       n     min     max
##   <int> <dbl> <dbl>
## 1   500    NA     NA
```

only this is for *all* the babies, premature or not.⁵ So we want it by prematurity, which means a `group_by` first:

```
bw %>% group_by(`Premie?`) %>%
  summarize( n=n(),
            min=min(`Weeks Gestation`),
            max=max(`Weeks Gestation`))

## # A tibble: 3 x 4
##   `Premie?`      n     min     max
##   <int> <int> <dbl> <dbl>
## 1      0  424    37    45
## 2      1   75    20    36
## 3     NA    1    NA     NA
```

`group_by` with a number works, even though using the number in ``Premie?`` in a boxplot didn't. `group_by` just uses the distinct values, whether they are numbers, text or factor levels.

Any of these graphs or summaries will help you answer the question, in the same way. The ultimate issue here is “something that will get the job done”: it doesn't matter so much what.

In R, NA means “missing”. When you try to compute something containing a missing value, the answer is usually missing (since you don't know what the missing value is). That's why the first `summarize` gave us missing values: there was one missing weeks of gestation in with all the ones for which we had values, so the max and min had to be missing as well. In the second `summarize`, the one by whether a baby was born prematurely or not, we learn a bit more about that missing ``Premie?``: evidently its weeks of gestation was missing as well, since the min and max of that were missing.⁶

Here's that baby. I'm doing a bit of fiddling to show all the columns (as rows, since there's only one actual row). Don't worry about the second line of code below; we will investigate that later.

```
bw %>% filter(is.na(`Premie?`)) %>%
  gather(name,value,everything())
```

```
## # A tibble: 10 x 2
##   name          value
##   <chr>         <dbl>
## 1 Father Age    33
## 2 Mother Age    32
## 3 Weeks Gestation NA
## 4 Pre-natal Visits 9
## 5 Marital Status 1
## 6 Mother Weight Gained 25
## 7 Low Birthweight? 0
## 8 Weight (pounds) 7.19
## 9 Premie?      NA
## 10 Few Visits?    0
```

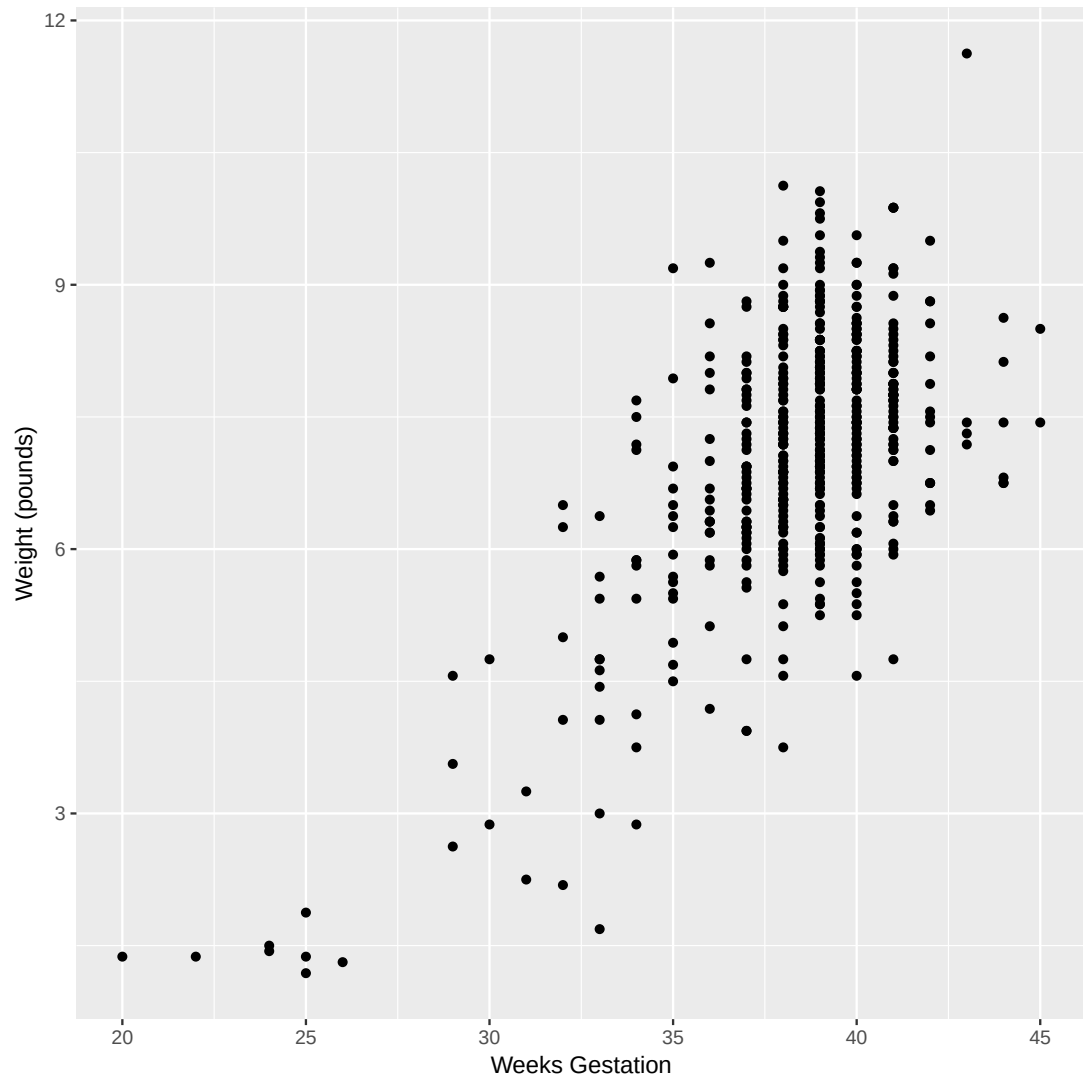
The *only* thing that was missing was its weeks of gestation, but that prevented anyone from figuring out whether it was premature or not.

- (b) Explore the relationship between birth weight and length of pregnancy (“gestation”) using a suitable graph. What do you see?

Solution:

This needs to be a scatterplot because these are both quantitative variables:

```
ggplot(bw,aes(x=`Weeks Gestation`,y=`Weight (pounds)`))+geom_point()
## Warning: Removed 1 rows containing missing values (geom_point).
```

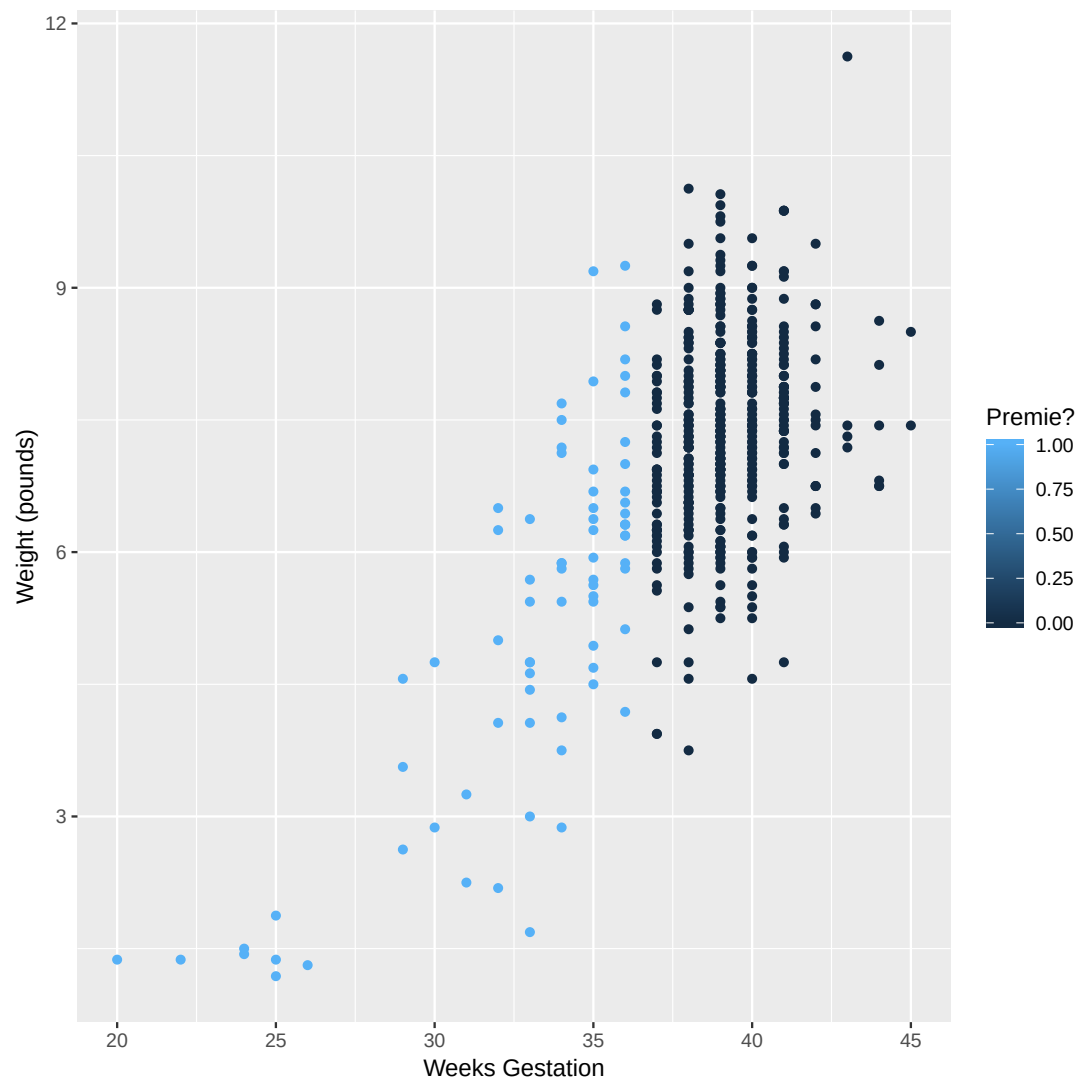


You see a rather clear upward trend. Those very underweight babies came from very short pregnancies, but the vast majority of pregnancies were of more or less normal length (40 weeks is normal) and resulted in babies of more or less normal birth weight.

I want to illustrate something else: how about *colouring* the births that were premature? Piece of cake with `ggplot`:

```
ggplot(bw,aes(x=`Weeks Gestation`,y=`Weight (pounds)`,
              colour=`Premie?`))+geom_point()
```

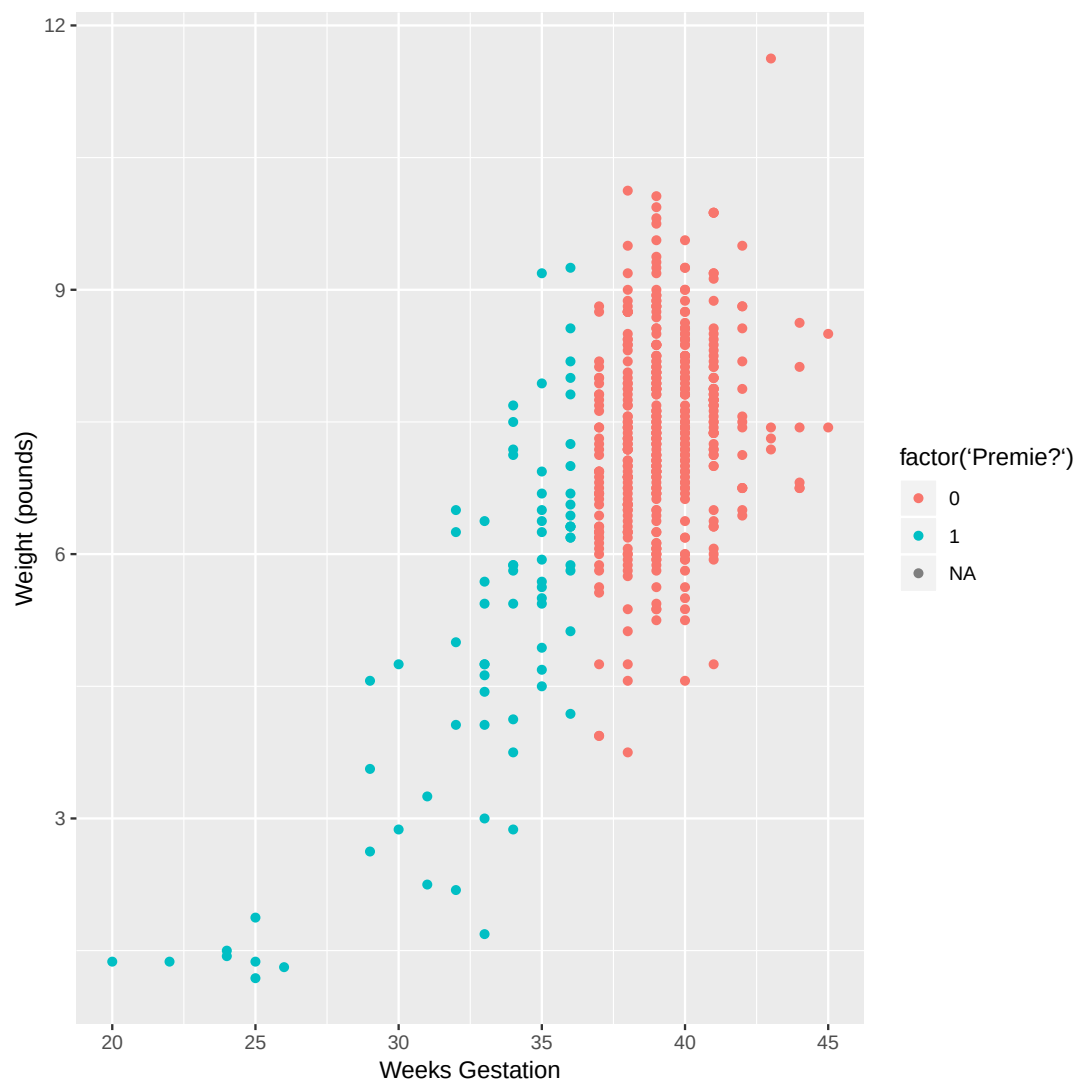
```
## Warning: Removed 1 rows containing missing values (geom_point).
```



That was rather silly because `ggplot` treated prematureness as a *continuous* variable, and plotted the values on a dark blue-light blue scale. This is the same issue as on the boxplot above, and has the same solution:

```
ggplot(bw,aes(x=`Weeks Gestation`,y=`Weight (pounds)`,
              colour=factor(`Premie?`)))+geom_point()

## Warning: Removed 1 rows containing missing values (geom_point).
```



Better.

With the normal-length pregnancies (red), there seems to be no relationship between length of pregnancy and birth weight, just a random variation. But with the premature births, a shorter pregnancy typically goes with a *lower* birth weight. This would be why the birth weights for the premature births were more variable.

- (c) Do a web search to find the standard (North American) definition of a premature birth. Does that correspond to what you saw in the data? Cite the website you used, for example by saying “according to URL, ...”, with URL replaced by the address of the website you found.

Solution: The website <http://www.mayoclinic.org/diseases-conditions/premature-birth/basics/definition/con-20020050> says that “a premature birth is one that occurs before the start of the 37th week of pregnancy”, which is exactly what we found. (Note that I am citing the webpage on which I found this, and I even made it into a link so that you can check it.)

The Mayo Clinic is a famous hospital system with locations in several US states, so I think we can trust what its website says.

This, believe it or not, is a further continuation of the same question. If you read in the data before, you can skip (a). The histogram part will also look familiar.

3. The data in file <http://www.utsc.utoronto.ca/~butler/c32/ncbirths.csv> are about 500 randomly chosen births of babies in North Carolina. There is a lot of information: not just the weight at birth of the baby, but whether the baby was born prematurely, the ages of the parents, whether the parents are married, how long (in weeks) the pregnancy lasted (this is called the “gestation”) and so on. We have seen these data before.

- (a) Read in the data from the file into R, bearing in mind what type of file it is.

Solution:

This is a .csv file (it came from a spreadsheet), so it needs reading in accordingly. Work directly from the URL (rather than downloading the file, unless you are working offline):

```
myurl="http://www.utsc.utoronto.ca/~butler/c32/ncbirths.csv"
bw=read_csv(myurl)

## Parsed with column specification:
## cols(
##   'Father Age' = col_integer(),
##   'Mother Age' = col_integer(),
##   'Weeks Gestation' = col_integer(),
##   'Pre-natal Visits' = col_integer(),
##   'Marital Status' = col_integer(),
##   'Mother Weight Gained' = col_integer(),
##   'Low Birthweighti' = col_integer(),
##   'Weight (pounds)' = col_double(),
##   'Premiei' = col_integer(),
##   'Few Visitsi' = col_integer()
## )
```

- (b) Find a 95% confidence interval for the mean birth weight of all babies born in North Carolina (of which these babies are a sample). At the end, you should state what the confidence interval is. Giving some output is necessary, but *not* enough by itself.

If your variable name has a space or other special character (like a question mark) in it, remember that you have to surround its name with backticks, as discussed the first time we looked at these data.

Solution: This:

```
t.test(bw$`Weight (pounds)`)

##
## One Sample t-test
##
## data: bw$`Weight (pounds)`
## t = 104.94, df = 499, p-value < 2.2e-16
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  6.936407 7.201093
## sample estimates:
## mean of x
##  7.06875
```

or (the same, but remember to match your brackets):

```
with(bw,t.test(`Weight (pounds)`))

##
## One Sample t-test
##
## data: Weight (pounds)
## t = 104.94, df = 499, p-value < 2.2e-16
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  6.936407 7.201093
## sample estimates:
## mean of x
##  7.06875
```

The confidence interval goes from 6.94 to 7.20 pounds.

There is an annoyance about `t.test`. Sometimes you can use `data=` with it, and sometimes not. When we do a two-sample *t*-test later, there is a “model formula” with a squiggle in it, and there we can use `data=`, but here not, so you have to use the dollar sign or the `with` to say which data frame to get things from. The distinction seems to be that *if you are using a model formula*, you can use `data=`, and if not, not.

This is one of those things that is a consequence of R’s history. The original `t.test` was without the model formula and thus without the `data=`, but the model formula got “retro-fitted” to it later. Since the model formula comes from things like regression, where `data=` is legit, that had to be retro-fitted as well. Or, at least, that’s my understanding.

- (c) Birth weights of babies born in the United States have a mean of 7.3 pounds. Is there any evidence that babies born in North Carolina are less heavy on average? State appropriate hypotheses, do your test, obtain a P-value and state your conclusion, in terms of the original data.

Solution: Null is that the population mean (the mean weight of all babies born in North Carolina) is $H_0 : \mu = 7.3$ pounds, and the alternative is that the mean is less: $H_a : \mu < 7.3$ pounds. This is a one-sided alternative, which we need to feed into `t.test`:

```
t.test(bw$`Weight (pounds)` ,mu=7.3,alternative="less")

##
## One Sample t-test
##
## data:  bw$`Weight (pounds)`
## t = -3.4331, df = 499, p-value = 0.0003232
## alternative hypothesis: true mean is less than 7.3
## 95 percent confidence interval:
##      -Inf 7.179752
## sample estimates:
## mean of x
##      7.06875
```

Or with `with`. If you see what I mean.

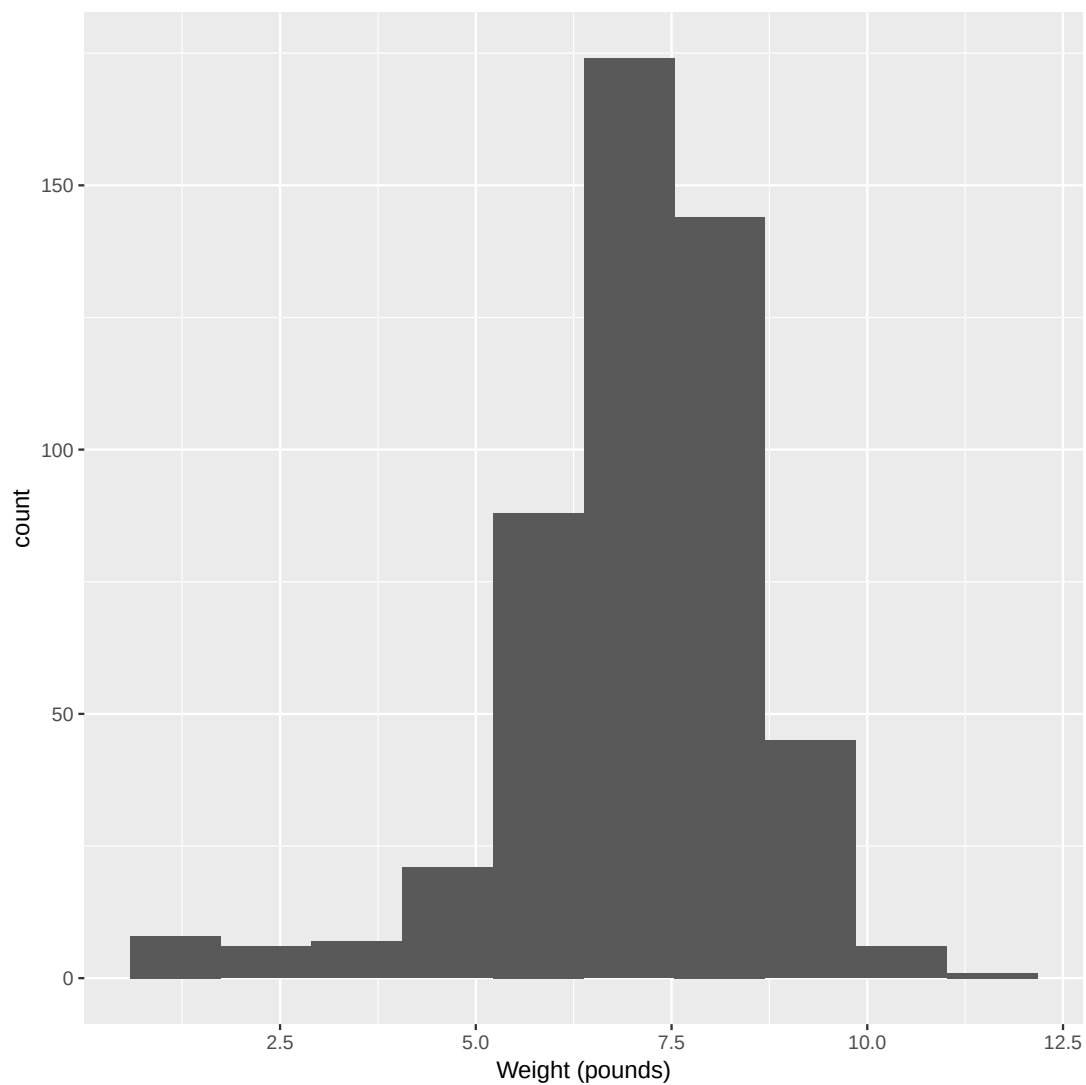
The P-value is 0.0003, which is *less* than any α we might have chosen: we *reject* the null hypothesis in favour of the alternative, and thus we conclude that the mean birth weight of babies in North Carolina is indeed less than 7.3 pounds.

“Reject the null hypothesis” is *not* a complete answer. You need to say something about what rejecting the null hypothesis means *in this case*: that is, you must make a statement about birth weights of babies.

- (d) The theory behind the *t*-test says that the distribution of birth weights should be (approximately) normally distributed. Obtain a histogram of the birth weights. Does it look approximately normal? Comment briefly. (You’ll have to pick a number of bins for your histogram first. I don’t mind very much what you pick, as long as it’s not obviously too many or too few bins.)

Solution: We did this before (and discussed the number of bins before), so I’ll just reproduce my 10-bin histogram (which is what I preferred, but this is a matter of taste):

```
ggplot(bw,aes(x=`Weight (pounds)`))+geom_histogram(bins=10)
```



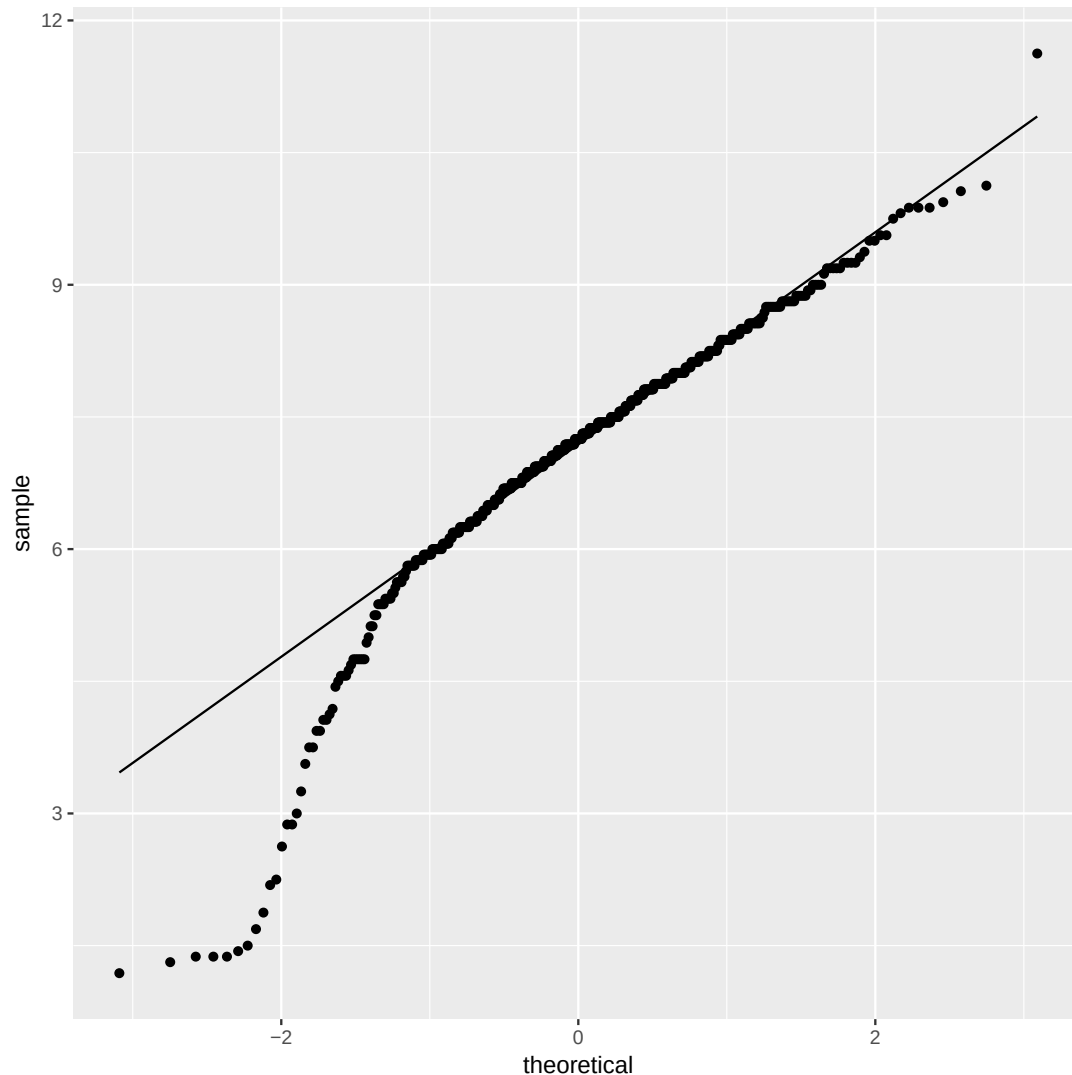
So, we were assessing normality. What about that?

It is mostly normal-looking, but I am suspicious about those *very* low birth weights, the ones below about 4 pounds. There are too many of those, as I see it.

If you think this is approximately normal, you need to make some comment along the lines of “the shape is approximately symmetric with no outliers”. I think my first answer is better, but this answer is worth something, since it is a not completely unreasonable interpretation of the histogram.

A normal quantile plot is better for assessing normality than a histogram is, but I won’t make you do one until we have seen the idea in class. Here’s the normal quantile plot for these data:

```
ggplot(bw,aes(sample=`Weight (pounds)`))+stat_qq()+stat_qq_line()
```



This is rather striking: the lowest birthweights (the ones below 5 pounds or so) are *way* too low for a normal distribution to apply. The top end is fine (except perhaps for that one very heavy baby), but there are too many low birthweights for a normal distribution to be believable. Note how much clearer this story is than on the histogram.

Having said that, the t -test, especially with a sample size as big as this (500), behaves *very* well when the data are somewhat non-normal (because it takes advantage of the Central Limit Theorem: that is, it's the *sampling distribution of the sample mean* whose shape matters). So, even though the data are definitely not normal, I wouldn't be too worried about our test.

This perhaps gives some insight as to why Freedman-Diaconis said we should use so many bins for our histogram. We have a lot of low-end outliers, so that the IQR is actually *small* compared to the overall spread of the data (as measured, say, by the SD or the range) and so FD thinks we need a lot of bins to describe the shape. Sturges is based on data being approximately normal, so it will tend to produce a small number of bins for data that have outliers.

4. Nenana, Alaska, is about 50 miles west of Fairbanks. Every spring, there is a contest in Nenana. A

wooden tripod is placed on the frozen river, and people try to guess the exact minute when the ice melts enough for the tripod to fall through the ice. The contest started in 1917 as an amusement for railway workers, and has taken place every year since. Now, hundreds of thousands of people enter their guesses on the Internet and the prize for the winner can be as much as \$300,000.

Because so much money is at stake, and because the exact same tripod is placed at the exact same spot on the ice every year, the data are consistent and accurate. The data are in <http://www.uts.utoronto.ca/~butler/c32/nenana.txt>.

- (a) Read the data into R. Note that the values are separated by *tabs* rather than spaces, so you'll need an appropriate `read_` to read it in.

Solution: These are “tab-separated values”, so `read_tsv` is the thing, as for the Australian athletes:

```
myurl="http://www.uts.utoronto.ca/~butler/c32/nenana.txt"
nenana=read_tsv(myurl)

## Parsed with column specification:
## cols(
##   Year = col_integer(),
##   JulianDate = col_double(),
##   'Date&Time' = col_character()
## )
```

Use whatever name you like for the data frame. One that is different from any of the column headers is smart; then it is clear whether you mean the whole data frame or one of its columns. `ice` or `melt` or anything like that would also be good.

I haven't asked you to display or check the data (that's coming up), but if you look at it and find that it didn't work, you'll know to come back and try this part again. R usually gets it right or gives you an error.

If you look at the data, they do appear to be separated by spaces, but the text version of the date and time *also* have spaces in them, so things might go astray if you try and read the values in without recognizing that the actual separator is a tab:

```
x=read_delim(myurl," ")

## Parsed with column specification:
## cols(
##   'Year JulianDate Date&Time' = col_character()
## )

## Warning in rbind(names(probs), probs_f): number of columns of result is not
a multiple of vector length (arg 1)

## Warning: 87 parsing failures.
## row # A tibble: 5 x 5 col      row col expected actual   file
<int> <chr> <chr>      <chr>      <chr>          actual 1
1 <NA> 1 columns 5 columns 'http://www.utsc.utoronto.ca/~butler/c3~ file 2    2
<NA> 1 columns 5 columns 'http://www.utsc.utoronto.ca/~butler/c3~ row 3    3 <NA>
1 columns 5 columns 'http://www.utsc.utoronto.ca/~butler/c3~ col 4    4 <NA> 1
columns 5 columns 'http://www.utsc.utoronto.ca/~butler/c3~ expected 5    5 <NA>
1 columns 5 columns 'http://www.utsc.utoronto.ca/~butler/c3~
## ... ..
.....
.....
.....
.....
.....
.....
## See problems(...) for more details.
```

Ouch! A hint as to what went wrong comes from looking at the read-in data frame:

```
x

## # A tibble: 87 x 1
##   `Year\tJulianDate\tDate&Time`
##   <chr>
## 1 "1917\t120.4795\tApril"
## 2 "1918\t131.3983\tMay"
## 3 "1919\t123.6066\tMay"
## 4 "1920\t132.4490\tMay"
## 5 "1921\t131.2795\tMay"
## 6 "1922\t132.5559\tMay"
## 7 "1923\t129.0837\tMay"
## 8 "1924\t132.6323\tMay"
## 9 "1925\t127.7726\tMay"
## 10 "1926\t116.6691\tApril"
## # ... with 77 more rows
```

Those `\t` symbols mean “tab character”, which is our hint that the values were separated by tabs rather than spaces.

More detail (if you can bear to see it) is here:

```
problems(x)

## # A tibble: 87 x 5
##   row col expected actual file
##   <int> <chr> <chr>    <chr> <chr>
## 1     1 <NA> 1 columns 5 colum~ 'http://www.utsc.utoronto.ca/~butler/c3~
## 2     2 <NA> 1 columns 5 colum~ 'http://www.utsc.utoronto.ca/~butler/c3~
## 3     3 <NA> 1 columns 5 colum~ 'http://www.utsc.utoronto.ca/~butler/c3~
## 4     4 <NA> 1 columns 5 colum~ 'http://www.utsc.utoronto.ca/~butler/c3~
## 5     5 <NA> 1 columns 5 colum~ 'http://www.utsc.utoronto.ca/~butler/c3~
## 6     6 <NA> 1 columns 5 colum~ 'http://www.utsc.utoronto.ca/~butler/c3~
## 7     7 <NA> 1 columns 5 colum~ 'http://www.utsc.utoronto.ca/~butler/c3~
## 8     8 <NA> 1 columns 5 colum~ 'http://www.utsc.utoronto.ca/~butler/c3~
## 9     9 <NA> 1 columns 5 colum~ 'http://www.utsc.utoronto.ca/~butler/c3~
## 10    10 <NA> 1 columns 5 colum~ 'http://www.utsc.utoronto.ca/~butler/c3~
## # ... with 77 more rows
```

The first line of the data file (with the variable names in it) had no spaces, only tabs, so `read_delim` thinks there is *one* column with a very long name, but in the actual data, there are *five* space-separated columns. The text date-times are of the form “April 30 at 11:30 AM”, which, if you think it’s all separated by spaces, is actually 5 things: April, 30, at and so on. These are the only things that are separated by spaces, so, from that point of view, there are five columns.

- (b) Find a way of displaying how many rows and columns your data frame has, and some of the values. Describe the first and last of the variables that you appear to have.

Solution: The easiest is just to display the tibble:

```
nenana

## # A tibble: 87 x 3
##   Year JulianDate `Date&Time`
##   <int>      <dbl> <chr>
## 1 1917      120. April 30 at 11:30 AM
## 2 1918      131. May 11 at 9:33 AM
## 3 1919      124. May 3 at 2:33 PM
## 4 1920      132. May 11 at 10:46 AM
## 5 1921      131. May 11 at 6:42 AM
## 6 1922      133. May 12 at 1:20 PM
## 7 1923      129. May 9 at 2:00 AM
## 8 1924      133. May 11 at 3:10 PM
## 9 1925      128. May 7 at 6:32 PM
## 10 1926      117. April 26 at 4:03 PM
## # ... with 77 more rows
```

Alternatively, you can take a `glimpse` of it:

```
glimpse(nenana)

## Observations: 87
## Variables: 3
## $ Year      <int> 1917, 1918, 1919, 1920, 1921, 1922, 1923, 1924, 19...
## $ JulianDate <dbl> 120.4795, 131.3983, 123.6066, 132.4490, 131.2795, ...
## $ `Date&Time` <chr> "April 30 at 11:30 AM", "May 11 at 9:33 AM", "May ...
```

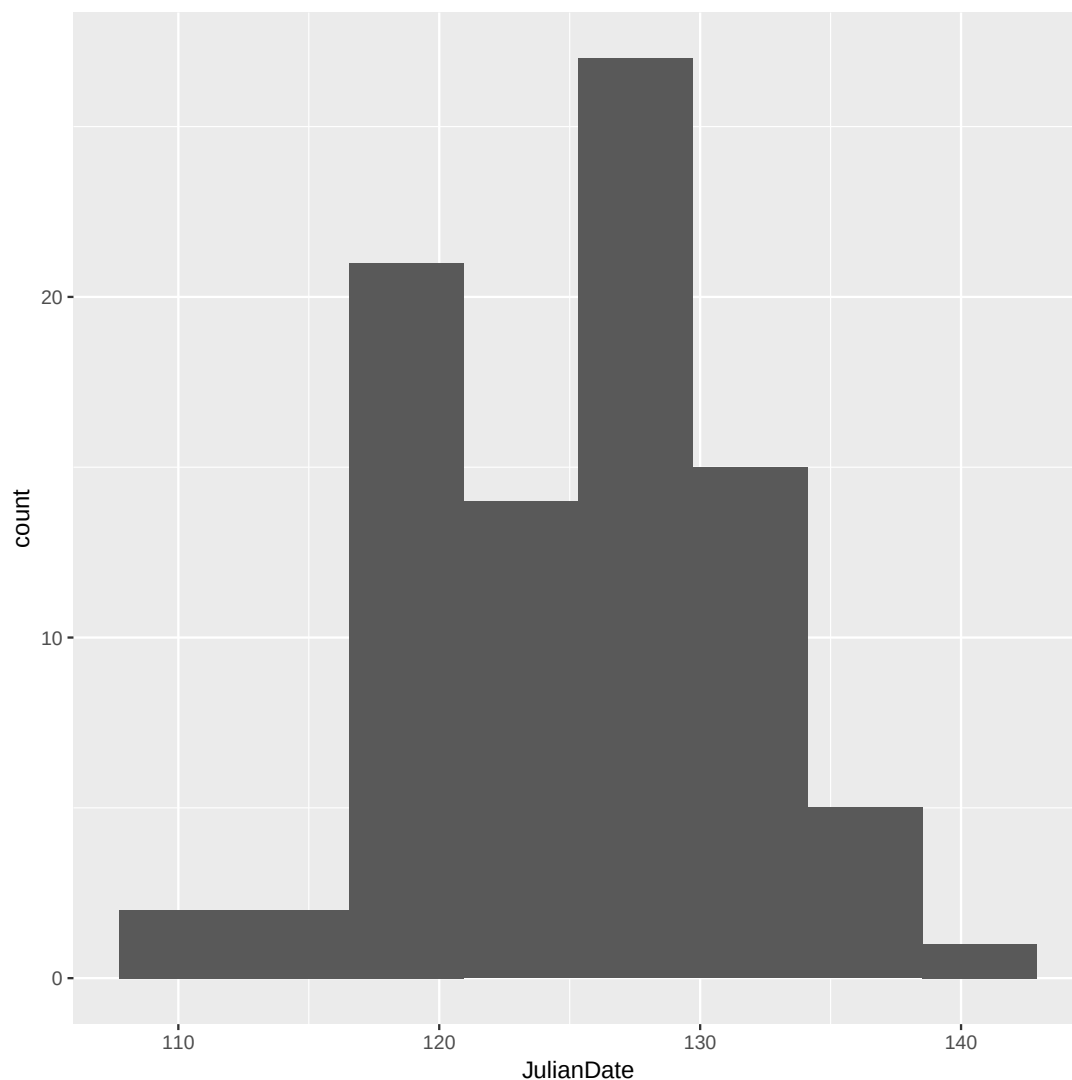
There are 87 years, and 3 columns (variables). The first column is year, and the last column is the date and time that the tripod fell into the river, written as a piece of text. I explain the second column in a moment.

- (c) Dates and times are awkward to handle with software. (We see more ways later in the course.) The column **JulianDate** expresses the time that the tripod fell through the ice as a fractional number of days since December 31. This enables the time (as a fraction of the way through the day) to be recorded as well, the whole thing being an ordinary number.

Make a histogram of the Julian dates. Comment briefly on its shape.

Solution: With a **ggplot** histogram, we need a number of bins first. I can do Sturges' rule in my head: the next power of 2 up from 87 (our n) is 128, which is 2^7 , so the base 2 log of 87 rounds up to 7. That plus one is 8, so we need 8 bins. For you, any not-insane number of bins will do, or any not-insane bin width, if you want to go that way:

```
ggplot(nenana, aes(x=JulianDate))+geom_histogram(bins=8)
```

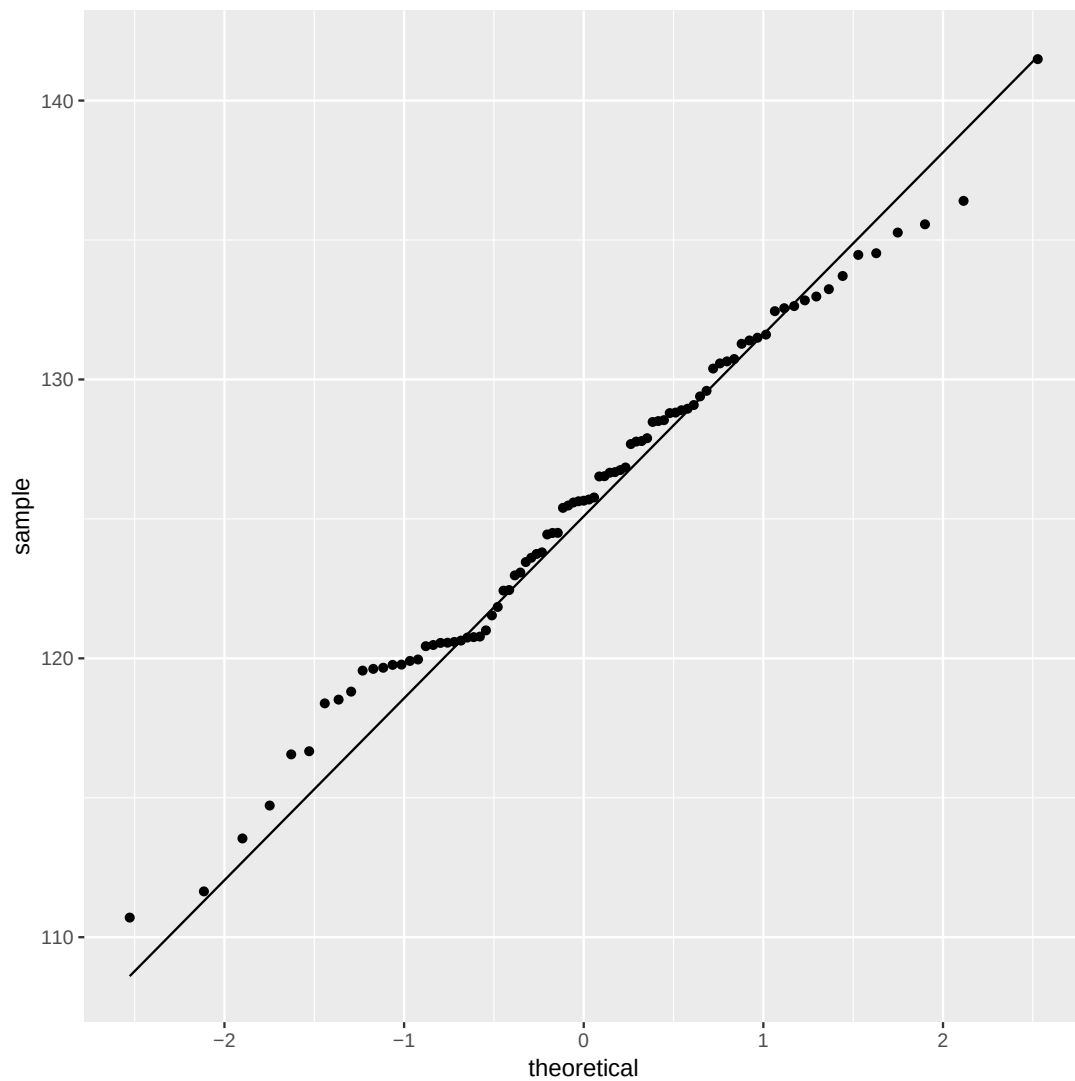


Note that you need to type `JulianDate` exactly as it appears, capital letters and all. R is case-sensitive.

This histogram looks more or less symmetric (and, indeed, normal). I really don't think you can justify an answer other than "symmetric" here. (Or "approximately normal": that's good too.) If your histogram is different, say so. I think that "hole" in the middle is not especially important.

We haven't done normal quantile plots yet, but looking ahead:

```
ggplot(nenana, aes(sample=JulianDate))+stat_qq()+stat_qq_line()
```



That hugs the line pretty well, so I would call it close to normally-distributed. It bulges away from the line because there are more values just below 120 than you would expect for a normal. This corresponds to the histogram bar centred just below 120 being taller than you would have expected.⁷

Extra: looking *way* ahead (to almost the end of the R stuff), this is how you handle the dates and times:

```

library(lubridate)

##
## Attaching package: 'lubridate'
##
## The following object is masked from 'package:base':
##
##     date
nenana %>%
  mutate(longdt=str_c(Year," ",`Date&Time`)) %>%
  mutate(datetime=ymd_hm(longdt,tz="America/Anchorage"))

## # A tibble: 87 x 5
##   Year JulianDate `Date&Time`      longdt      datetime
##   <int>      <dbl> <chr>          <chr>      <dtm>
## 1 1917        120. April 30 at 11:3~ 1917 April 30 a~ 1917-04-30 11:30:00
## 2 1918        131. May 11 at 9:33 AM 1918 May 11 at ~ 1918-05-11 09:33:00
## 3 1919        124. May 3 at 2:33 PM 1919 May 3 at 2~ 1919-05-03 14:33:00
## 4 1920        132. May 11 at 10:46 ~ 1920 May 11 at ~ 1920-05-11 10:46:00
## 5 1921        131. May 11 at 6:42 AM 1921 May 11 at ~ 1921-05-11 06:42:00
## 6 1922        133. May 12 at 1:20 PM 1922 May 12 at ~ 1922-05-12 13:20:00
## 7 1923        129. May 9 at 2:00 AM 1923 May 9 at 2~ 1923-05-09 02:00:00
## 8 1924        133. May 11 at 3:10 PM 1924 May 11 at ~ 1924-05-11 15:10:00
## 9 1925        128. May 7 at 6:32 PM 1925 May 7 at 6~ 1925-05-07 18:32:00
## 10 1926        117. April 26 at 4:03~ 1926 April 26 a~ 1926-04-26 16:03:00
## # ... with 77 more rows

```

I am not doing any further analysis with these, so just displaying them is good.

I have to do a preliminary step to get the date-times *with* their year in one place. `str_c` glues pieces of text together: in this case, the year, a space, and then the rest of the `Date&Time`. I stored this in `longdt`. The second `mutate` is the business end of it: `ymd_hm` takes a piece of text containing a year, month (by name or number), day, hours, minutes *in that order*, and extracts those things from it, storing the whole thing as an R date-time. Note that the AM/PM was handled properly. The benefit of doing that is we can extract anything from the dates, such as the month or day of week, or take differences between the dates. Or even check that the Julian dates were calculated correctly (the `lubridate` function is called `yday` for "day of year"):

```

nenana %>%
  mutate(longdt=str_c(Year," ",`Date&Time`)) %>%
  mutate(datetime=ymd_hm(longdt,tz="America/Anchorage")) %>%
  mutate(jd=yday(datetime)) ->
nenana2
nenana2 %>% select(JulianDate,jd,datetime)

## # A tibble: 87 x 3
##   JulianDate    jd datetime
##   <dbl> <dbl> <dtm>
## 1      120.   120 1917-04-30 11:30:00
## 2      131.   131 1918-05-11 09:33:00
## 3      124.   123 1919-05-03 14:33:00
## 4      132.   132 1920-05-11 10:46:00
## 5      131.   131 1921-05-11 06:42:00
## 6      133.   132 1922-05-12 13:20:00
## 7      129.   129 1923-05-09 02:00:00
## 8      133.   132 1924-05-11 15:10:00
## 9      128.   127 1925-05-07 18:32:00
## 10     117.   116 1926-04-26 16:03:00
## # ... with 77 more rows

```

Hmm, some of those are off by one. What do the off-by-one ones have in common? Let's look at more of them. `round` rounds off to the nearest integer (since these are actually decimal numbers):

```

nenana2 %>%
  filter(round(JulianDate) != round(jd)) %>%
  select(JulianDate,jd,datetime)

## # A tibble: 61 x 3
##   JulianDate    jd datetime
##   <dbl> <dbl> <dtm>
## 1      124.   123 1919-05-03 14:33:00
## 2      133.   132 1922-05-12 13:20:00
## 3      133.   132 1924-05-11 15:10:00
## 4      128.   127 1925-05-07 18:32:00
## 5      117.   116 1926-04-26 16:03:00
## 6      128.   127 1928-05-06 16:25:00
## 7      126.   125 1929-05-05 15:41:00
## 8      129.   128 1930-05-08 19:03:00
## 9      129.   128 1933-05-08 19:30:00
## 10     121.   120 1934-04-30 14:07:00
## # ... with 51 more rows

```

The ones shown here are all *after noon*, and the Julian date in the data file appears as one more than the one calculated by `lubridate`. What has actually happened is a quirk of how tibbles are displayed: they show 3 significant digits, *rounded*. The Julian dates given by `yday` are the whole-number part, so the ones in the data value are that plus more than 0.5, which will round *up*. The first line of code below displays 6 significant digits rather than only three:

```
options(pillar.sigfig=6)
nenana2 %>%
  filter(round(JulianDate) != round(jd)) %>%
  select(JulianDate,jd,datetime)

## # A tibble: 61 x 3
##   JulianDate    jd datetime
##   <dbl> <dbl> <dtm>
## 1 123.607 123 1919-05-03 14:33:00
## 2 132.556 132 1922-05-12 13:20:00
## 3 132.632 132 1924-05-11 15:10:00
## 4 127.773 127 1925-05-07 18:32:00
## 5 116.669 116 1926-04-26 16:03:00
## 6 127.684 127 1928-05-06 16:25:00
## 7 125.654 125 1929-05-05 15:41:00
## 8 128.794 128 1930-05-08 19:03:00
## 9 128.813 128 1933-05-08 19:30:00
## 10 120.588 120 1934-04-30 14:07:00
## # ... with 51 more rows
```

Displaying more decimals shows that I was right: `jd` is (to this accuracy) a whole number, but `JulianDate` is a decimal with fractional part greater than 0.50.

Now I have to turn the extra significant digits off:

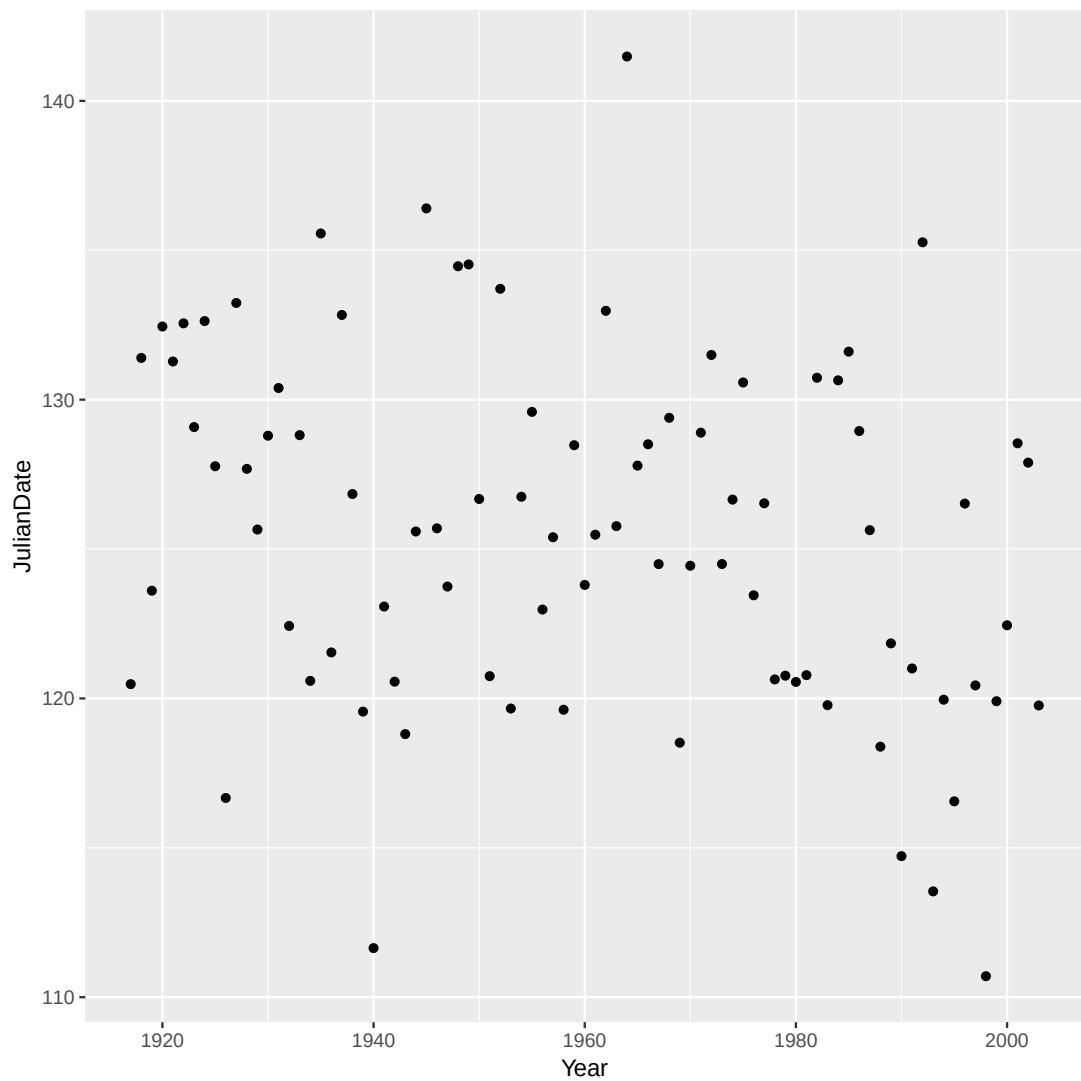
```
options(pillar.sigfig=3)
```

- (d) Plot `JulianDate` against `Year` on a scatterplot. What recent trends, if any, do you see? Comment briefly.

Solution:

`geom_point`:

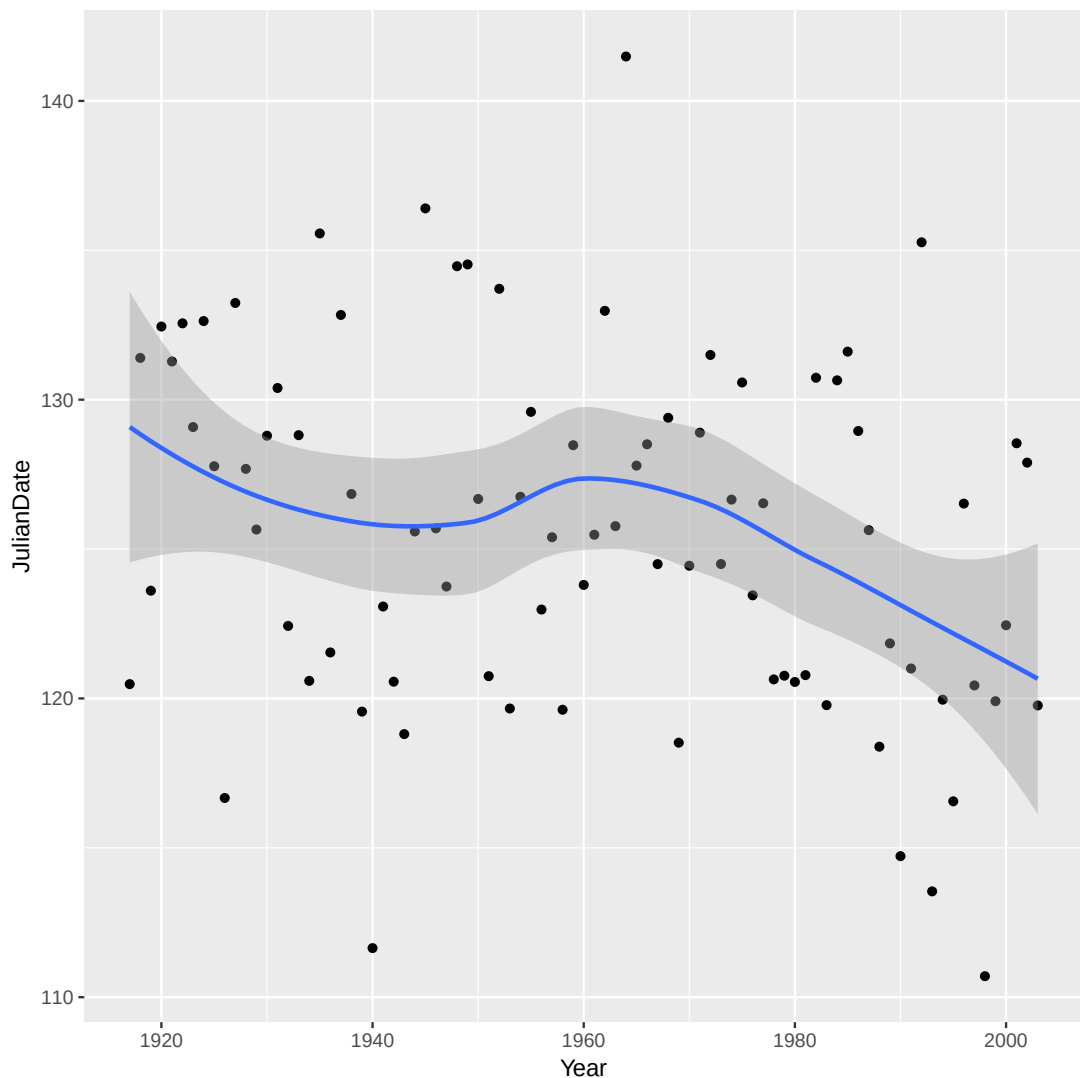
```
ggplot(nenana,aes(x=Year,y=JulianDate))+geom_point()
```



This is actually a small-but-real downward trend, especially since about 1960, but the large amount of variability makes it hard to see, so I'm good with either "no trend" or "weak downward trend" or anything roughly like that. There is definitely not much trend before 1960, but most of the really early break-ups (less than about 118) have been since about 1990.

You can even add to the `ggplot`, by putting a smooth trend on it:

```
ggplot(nenana, aes(x=Year, y=JulianDate)) + geom_point() + geom_smooth()
## 'geom_smooth()' using method = 'loess' and formula 'y ~ x'
```



This is R's version of a trend that is not constrained to be linear (so that it “lets the data speak for itself”).

Now there is something obvious to see: after about 1960, there is a clear downward trend: the ice is breaking up earlier on average every year. Even though there is a lot of variability, the overall trend, viewed this way, is clear.

What does this mean, in practice? This notion of the ice melting earlier than it used to is consistent all over the Arctic, and is one more indication of climate change. Precisely, it is an indication that climate change is happening, but we would have to delve further to make any statements about the *cause* of that climate change.

This is also a continuation of the previous question. If you read in the data before, use your data frame from there.

5. Nenana, Alaska, is about 50 miles west of Fairbanks. Every spring, there is a contest in Nenana. A wooden tripod is placed on the frozen river, and people try to guess the exact minute when the ice melts enough for the tripod to fall through the ice. The contest started in 1917 as an amusement for railway

workers, and has taken place every year since. Now, hundreds of thousands of people enter their guesses on the Internet and the prize for the winner can be as much as \$300,000.

Because so much money is at stake, and because the exact same tripod is placed at the exact same spot on the ice every year, the data are consistent and accurate. The data are in <http://www.uts.utoronto.ca/~butler/c32/nenana.txt>.

Yes, we saw these data before.

- (a) Read the data into R, as before, or use the data frame that you read in before. Note that the values are separated by *tabs* rather than spaces, so you'll need an appropriate `read_` to read it in.

Solution: These are “tab-separated values”, so `read_tsv` is the thing, as for the Australian athletes:

```
myurl="http://www.uts.utoronto.ca/~butler/c32/nenana.txt"
nenana=read_tsv(myurl)

## Parsed with column specification:
## cols(
##   Year = col_integer(),
##   JulianDate = col_double(),
##   'Date&Time' = col_character()
## )
```

Use whatever name you like for the data frame. One that is different from any of the column headers is smart; then it is clear whether you mean the whole data frame or one of its columns. `ice` or `melt` or anything like that would also be good.

- (b) Obtain a 90% confidence interval for the mean `JulianDate`. What interval do you get? Looking back at your histogram, do you have any doubts about the validity of what you have just done?

Solution: This is a matter of using `t.test` and pulling out the interval. Since we are looking for a non-standard interval, we have to remember `conf.level` as the way to get the confidence level that we want. I'm going with `with` this time, though the dollar-sign thing is equally as good:

```
with(nenana,t.test(JulianDate,conf.level=0.90))

##
## One Sample t-test
##
## data: JulianDate
## t = 197.41, df = 86, p-value < 2.2e-16
## alternative hypothesis: true mean is not equal to 0
## 90 percent confidence interval:
## 124.4869 126.6018
## sample estimates:
## mean of x
## 125.5443
```

Between 124.5 and 126.6 days into the year. Converting that into something we can understand (because I want to), there are $31 + 28 + 31 + 30 = 120$ days in January through April (in a non-leap year), so this says that the mean breakup date is between about May 4 and May 6.

The t -test is based on an assumption of data coming from a normal distribution. The histogram we made earlier looks pretty much normal, so there are no doubts about normality and thus no doubts about the validity of what we have done, on the evidence we have seen so far. (I have some doubts on different grounds, based on another of the plots we did earlier, which I'll explain later, but all I'm expecting you to do is to look at the histogram and say "Yep, that's normal enough". Bear in mind that the sample size is 87, which is large enough for the Central Limit Theorem to be pretty helpful, so that we don't need the data to be more than "approximately normal" for the sampling distribution of the sample mean to be very close to t with the right df.)

- (c) An old-timer in Nenana strokes his grey beard and says "When I were young, I remember the tripod used to fall into the water around May 10". In a non-leap year, May 10 is Julian day 130. Test the null hypothesis that the mean `JulianDay` is 130, against the alternative that it is less. What do you conclude? What practical implication does that have (assuming that the old-timer has a good memory)?

Solution: The test is `t.test` again, but this time we have to specify a null mean and a direction of alternative:

```
with(nenana,t.test(JulianDate,mu=130,alternative="less"))

##
##  One Sample t-test
##
## data:  JulianDate
## t = -7.0063, df = 86, p-value = 2.575e-10
## alternative hypothesis: true mean is less than 130
## 95 percent confidence interval:
##      -Inf 126.6018
## sample estimates:
## mean of x
## 125.5443
```

For a test, look first at the P-value, which is 0.0000000002575: that is to say, the P-value is very small, definitely smaller than 0.05 (or any other α you might have chosen). So we *reject* the null hypothesis, and conclude that the mean `JulianDate` is actually *less* than 130.

Now, this is the date on which the ice breaks up on average, and we have concluded that it is *earlier* than it used to be, since we are assuming the old-timer's memory is correct.

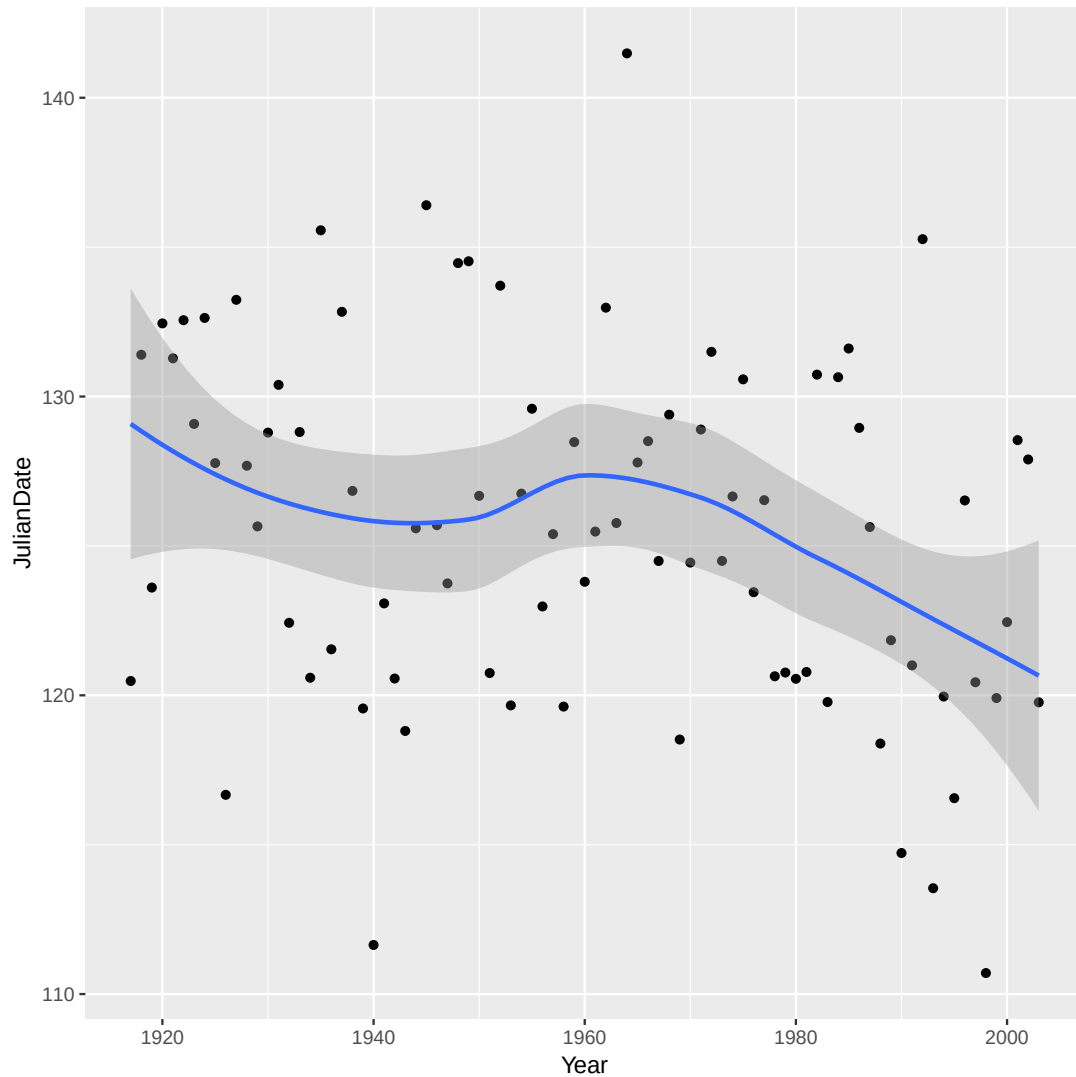
This is evidence in favour of global warming; a small piece of evidence, to be sure, but the ice is melting earlier than it used to all over the Arctic, so it's not just in Nenana that it is happening. You don't need to get to the "global warming" part, but I *do* want you to observe that the ice is breaking up earlier than it used to.

- (d) Plot `JulianDate` against `Year` on a scatterplot. What recent trends, if any, do you see? Comment briefly. (You did this before, but I have some extra comments on the graph this time, so feel free to just read this part.)

Solution:

I liked the `ggplot` with a smooth trend on it:

```
ggplot(nenana,aes(x=Year,y=JulianDate))+geom_point()+geom_smooth()  
## 'geom_smooth()' using method = 'loess' and formula 'y ~ x'
```

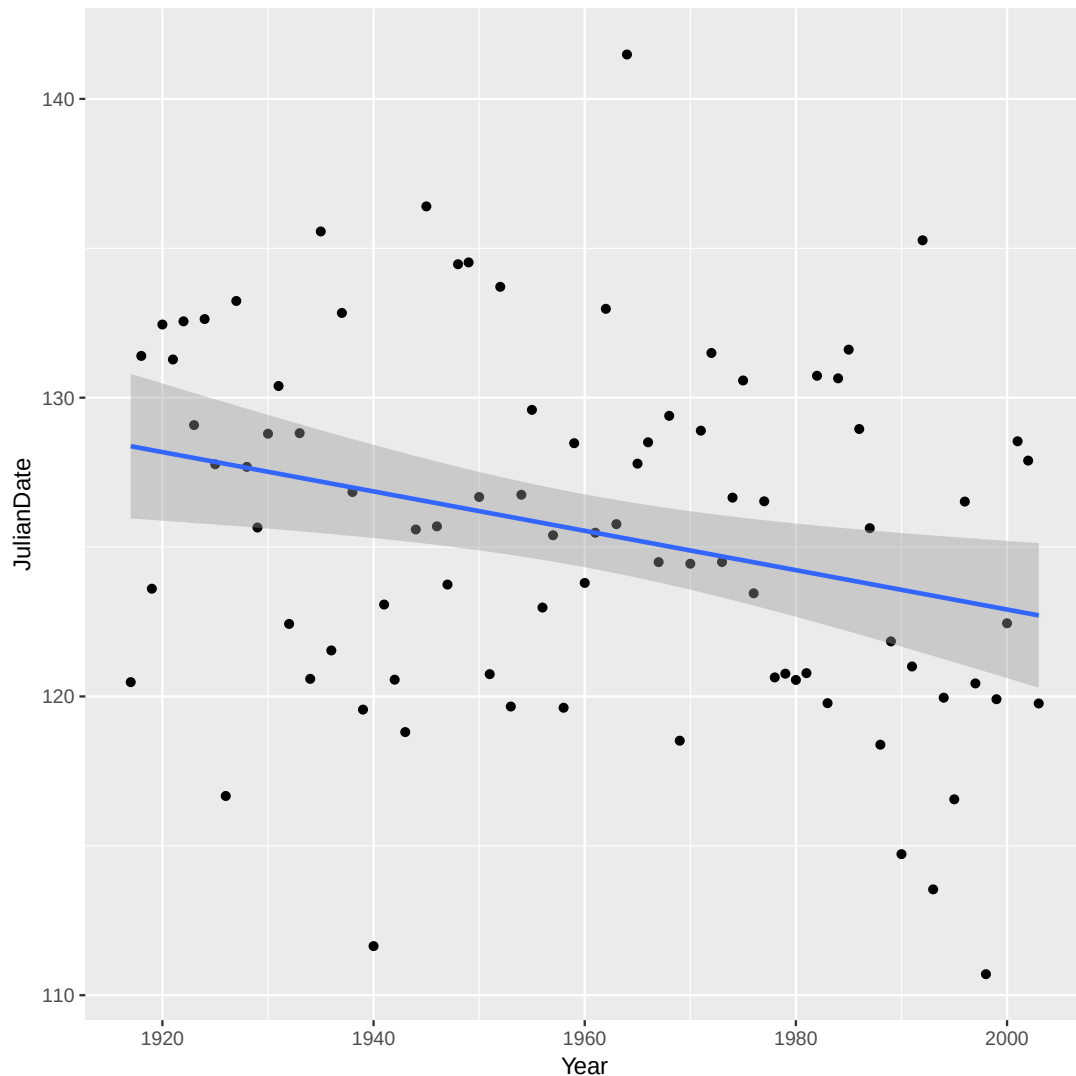


There was something obvious to see: after about 1960, there is a clear downward trend: the ice is breaking up earlier on average every year. Even though there is a lot of variability, the overall trend, viewed this way, is clear (and consistent with the test we did earlier). Note that the old-timer's value of 130 is the kind of **JulianDate** we would typically observe around 1920, which would make the old-timer over 90 years old.

All right, why did I say I had some doubts earlier? Well, because of this downward trend, the mean is not actually the same all the way through, so it doesn't make all that much sense to estimate it, which is what we were doing earlier by doing a confidence interval or a hypothesis test. What would actually make more sense is to estimate the mean **JulianDate** for a particular year. This could be done by a regression: predict **JulianDate** from **Year**, and then get a "confidence interval for the mean response" (as you would have seen in B27 or will see in C67). The trend isn't really linear, but is not that far off. I can modify the previous picture to

give you an idea. Putting in `method="lm"` fits a line; as we see later, `lm` does regressions in R:

```
ggplot(nenana,aes(x=Year,y=JulianDate))+geom_point()+  
  geom_smooth(method="lm")
```



Compare the confidence interval for the mean **JulianDate** in 1920: 126 to 131 (the shaded area on the graph), with 2000: 121 to 125. A change of about 5 days over 80 years. And with the recent trend that we saw above, it's probably changing faster than that now. Sobering indeed.

6. Beginning accounting students need to learn to audit in a computerized environment. A sample of beginning accounting students took each of two tests: the Computer Attitude Scale (CAS) and the Computer Anxiety Rating Scale (CARS). A higher score in each indicates greater anxiety around computers. The test scores are scaled to be between 0 and 5. Also noted was each student's gender. The data are in <http://www.utsc.utoronto.ca/~butler/c32/compatt.txt>. The data values are separated by spaces.

(a) Read the data into R. Do you have what you expected? Explain briefly.

Solution: Read in and display the data. This, I think, is the easiest way.

```
url="http://www.utoronto.ca/~butler/c32/compatt.txt"
anxiety=read_delim(url, " ")

## Parsed with column specification:
## cols(
##   gender = col_character(),
##   CAS = col_double(),
##   CARS = col_double()
## )

anxiety

## # A tibble: 35 x 3
##   gender  CAS  CARS
##   <chr> <dbl> <dbl>
## 1 female 2.85  2.9
## 2 male  2.6   2.32
## 3 female 2.2   1
## 4 male  2.65  2.58
## 5 male  2.6   2.58
## 6 male  3.2   3.05
## 7 male  3.65  3.74
## 8 female 2.55  1.9
## 9 male  3.15  3.32
## 10 male  2.8   2.74
## # ... with 25 more rows
```

There is a total of 35 students with a CAS score, a CARS score and a gender recorded for each. This is in line with what I was expecting. (You can also note that the genders appear to be a mixture of males and females.)

(b) How many males and females were there in the sample?

Solution: Most easily count:

```
anxiety %>% count(gender)

## # A tibble: 2 x 2
##   gender      n
##   <chr> <int>
## 1 female    15
## 2 male     20
```

This also works (and is therefore good):

```
anxiety %>% group_by(gender) %>% summarize(count=n())

## # A tibble: 2 x 2
##   gender count
##   <chr> <int>
## 1 female    15
## 2 male     20
```

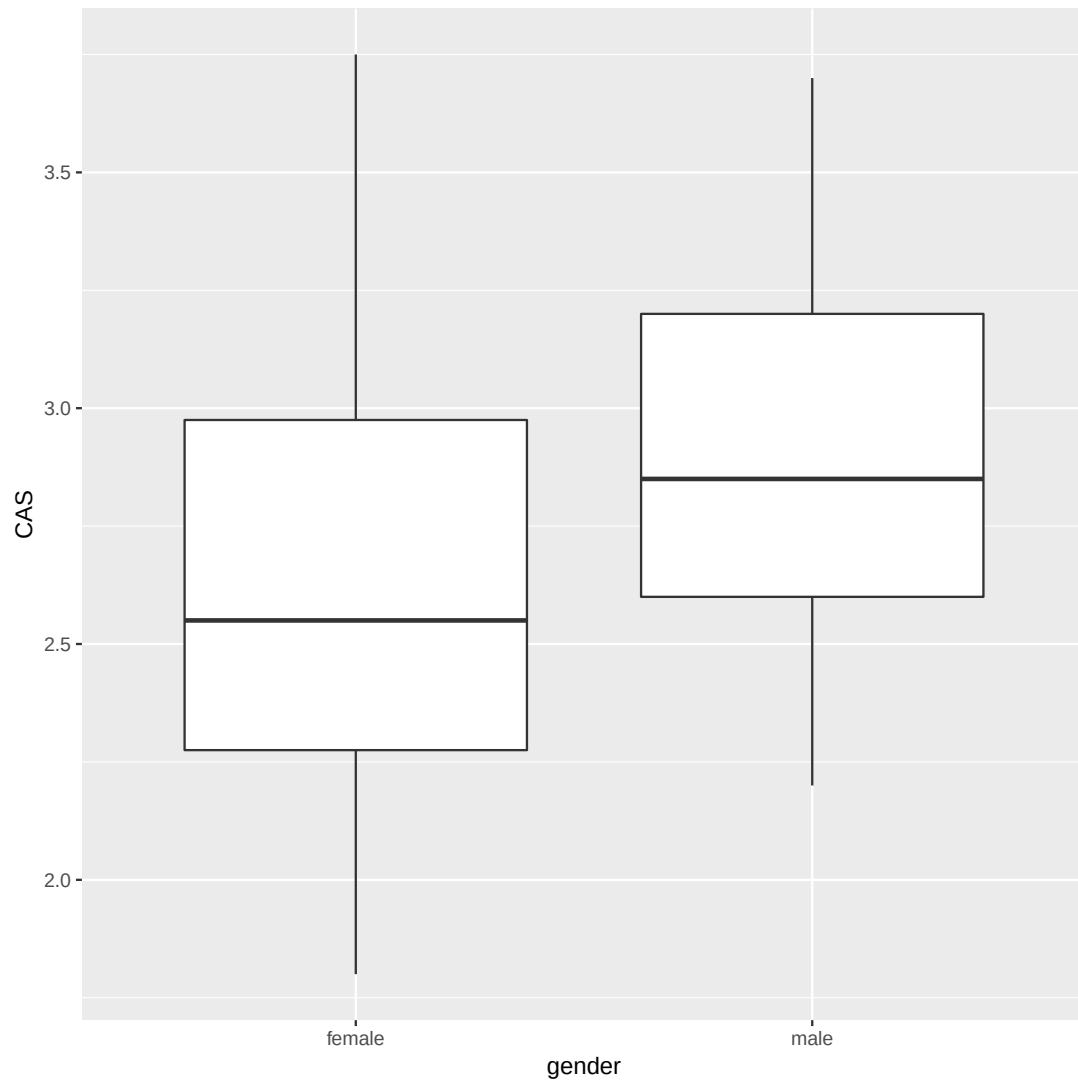
I want you to use R to do the counting (that is, don't just list out the whole data set with `print(n=Inf)` and count the males and females yourself). This is because you might have thousands of data values and you need to learn how to get R (or, later, SAS) to count them for you.

15 females and 20 males, *which you should say*. I made a point of *not* saying that it is enough to get the output with the answers on it, so you need to tell me what the answer is.

- (c) Do the CAS scores tend to be higher for females or for males? Draw a suitable graph to help you decide, and come to a conclusion.

Solution: Gender is categorical and CAS score is quantitative, so a boxplot would appear to be the thing:

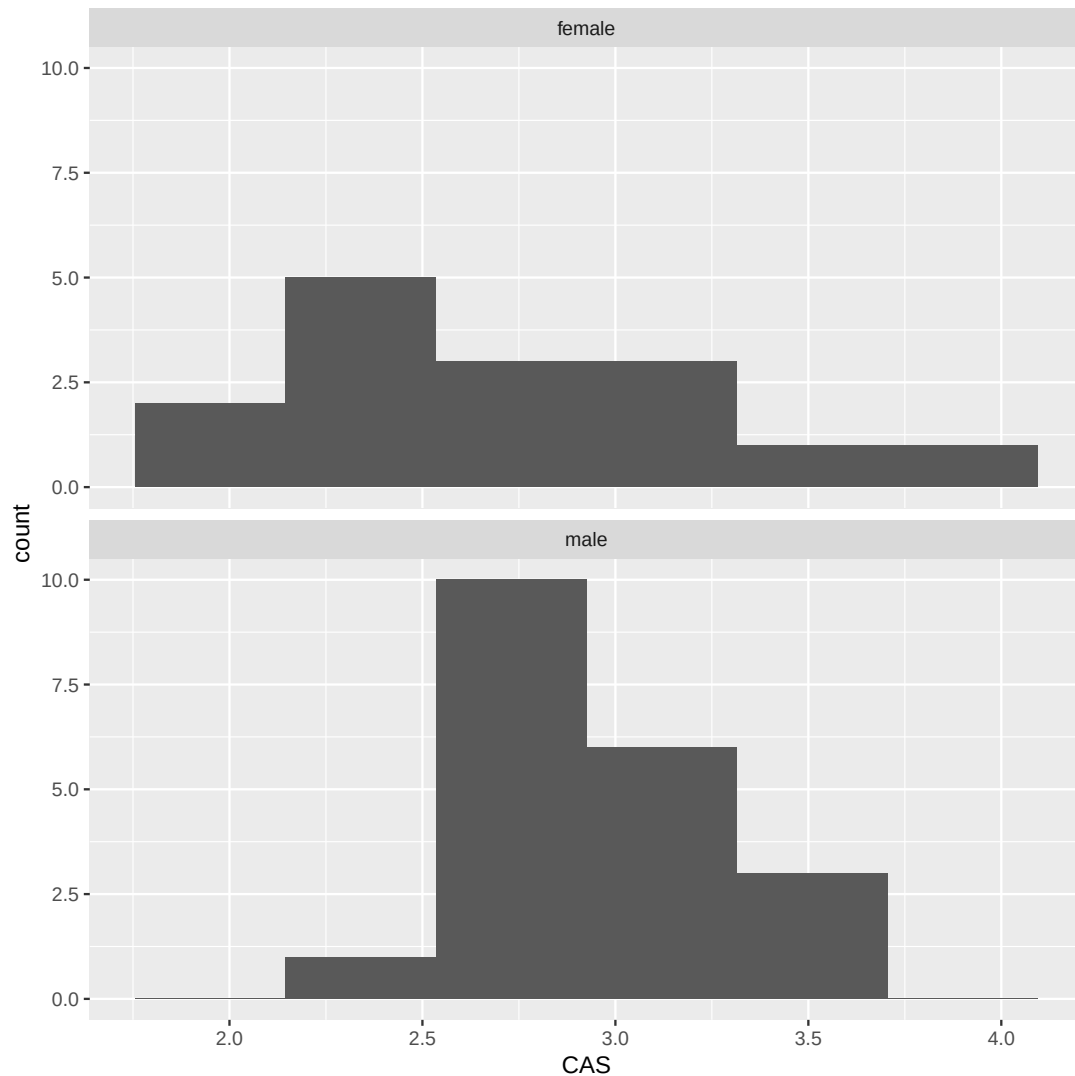
```
ggplot(anxiety, aes(x=gender, y=CAS)) + geom_boxplot()
```



The median for males is slightly higher, so male accountants are more anxious around computers than female accountants are.

If you wish, side-by-side (or, better, above-and-below) histograms would also work:

```
ggplot(anxiety, aes(x=CAS)) + geom_histogram(bins=6) +  
  facet_wrap(~gender, ncol=1)
```



If you go this way, you have to make a call about where the centres of the histograms are. I guess the male one is slightly further to the right, but it's not so easy to tell. (Make a call.)

- (d) Find the median CAS scores for each gender. Does this support what you saw on your plot? Explain briefly.

Solution: Group-by and summarize:

```
anxiety %>% group_by(gender) %>% summarize(med=median(CAS))

## # A tibble: 2 x 2
##   gender    med
##   <chr>   <dbl>
## 1 female  2.55
## 2 male   2.85
```

The median is a bit higher for males, which is what I got on my boxplot (and is apparently the same thing as is on the histograms, but it's harder to be sure there).

- (e) Find the mean and standard deviation of both CAS and CARS scores (for all the students combined, ie. not separated by gender) *without* naming those columns explicitly.

Solution: Without naming them explicitly means using some other way to pick them out of the data frame, either `summarize_if` or `summarize_at`. To do it the first way, ask what these two columns have in common: they are the only two numeric (quantitative) columns:

```
anxiety %>% summarize_if(is.numeric, funs(mean, sd))

## # A tibble: 1 x 4
##   CAS_mean CARS_mean CAS_sd CARS_sd
##   <dbl>    <dbl>   <dbl>   <dbl>
## 1     2.82     2.77  0.484   0.671
```

Or the second way, asking yourself what the *names* of those columns have in common: they start with C and the gender column doesn't:

```
anxiety %>% summarize_at(vars(starts_with("C")), funs(mean, sd))

## # A tibble: 1 x 4
##   CAS_mean CARS_mean CAS_sd CARS_sd
##   <dbl>    <dbl>   <dbl>   <dbl>
## 1     2.82     2.77  0.484   0.671
```

Either of these is good, or anything equivalent (like noting that the two anxiety scales both *ends_with* S):

```
anxiety %>% summarize_at(vars(ends_with("S")), funs(mean, sd))

## # A tibble: 1 x 4
##   CAS_mean CARS_mean CAS_sd CARS_sd
##   <dbl>    <dbl>   <dbl>   <dbl>
## 1     2.82     2.77  0.484   0.671
```

Because I didn't say otherwise, you should tell me what the means and SDs are, rounding off suitably: the CAS scores have mean 2.82 and SD 0.48, and the CARS scores have mean 2.77 and SD 0.67.

Yet another way to do it is to select the columns you want first (which you can do by number so as not to name them), and then find the mean and SD of all of them. This uses two tools that we haven't seen yet:

```
anxiety %>% select(2:3) %>% summarize_all(funs(mean,sd))

## # A tibble: 1 x 4
##   CAS_mean CARS_mean CAS_sd CARS_sd
##   <dbl>    <dbl>    <dbl>    <dbl>
## 1     2.82     2.77    0.484    0.671
```

This doesn't work:

```
summary(anxiety)

##      gender          CAS          CARS
## Length:35      Min.   :1.800      Min.   :1.000
## Class :character 1st Qu.:2.575      1st Qu.:2.445
## Mode  :character Median :2.800      Median :2.790
##              Mean  :2.816      Mean  :2.771
##              3rd Qu.:3.150      3rd Qu.:3.290
##              Max.  :3.750      Max.  :4.000
```

because, although it gets the means, it does not get the standard deviations. (I added the SD to the original question to make you find a way other than this.)

In summary, find a way to get those answers without naming those columns in your code, and I'm good.

In case you were wondering about how to do this separately by gender, well, put the `group_by` in like you did before:

```
anxiety %>% group_by(gender) %>%
  summarize_if(is.numeric,funs(mean,sd))

## # A tibble: 2 x 5
##   gender CAS_mean CARS_mean CAS_sd CARS_sd
##   <chr>    <dbl>    <dbl>    <dbl>    <dbl>
## 1 female  2.64      2.51    0.554    0.773
## 2 male   2.94      2.96    0.390    0.525
```

or

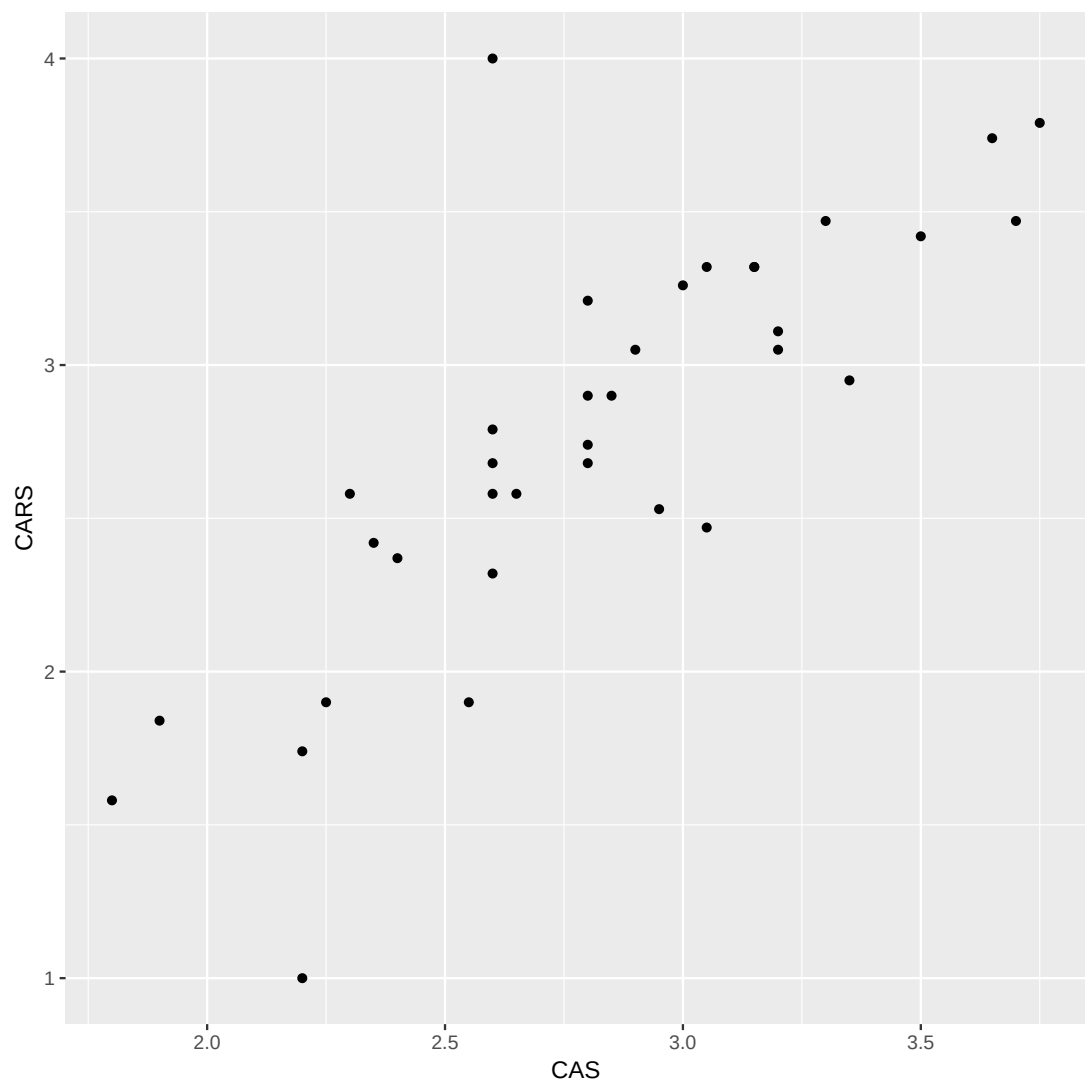
```
anxiety %>% group_by(gender) %>%
  summarize_at(vars(starts_with("C")),funs(mean,sd))

## # A tibble: 2 x 5
##   gender CAS_mean CARS_mean CAS_sd CARS_sd
##   <chr>    <dbl>    <dbl>    <dbl>    <dbl>
## 1 female  2.64      2.51    0.554    0.773
## 2 male   2.94      2.96    0.390    0.525
```

The male means are slightly higher on both tests, but the male standard deviations are a little smaller.

You might be wondering whether the test scores are related. They are both quantitative, so the obvious way to find out is a scatterplot:

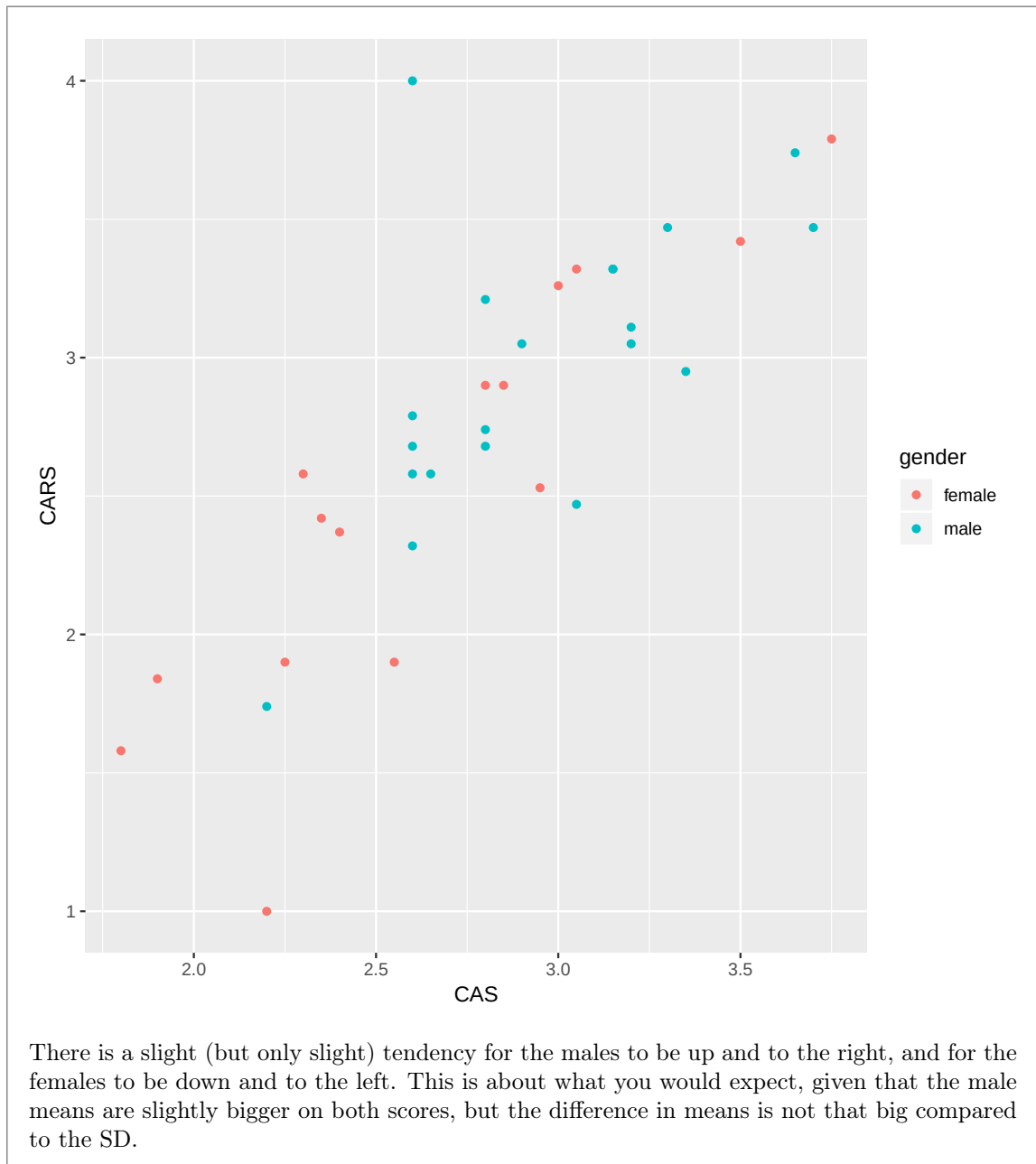
```
ggplot(anxiety,aes(x=CAS,y=CARS))+geom_point()
```



The two variables can be on either axis, since there is no obvious response or explanatory variable. A higher score on one scale goes with a higher score on the other, suggesting that the two scales are measuring the same thing.

This plot mixes up the males and females, so you might like to distinguish them, which goes like this:

```
ggplot(anxiety,aes(x=CAS,y=CARS,colour=gender))+geom_point()
```



7. A hunter-gatherer society is one where people get their food by hunting, fishing or foraging rather than by agriculture or by raising animals. Such societies tend to move from place to place. Anthropologists have studied hunter-gatherer societies in forest ecosystems across the world. The average population density of these societies is 7.38 people per 100 km². Hunter-gatherer societies on different continents might have different population densities, possibly because of large-scale ecological constraints (such as resource availability), or because of other factors, possibly social and/or historic, determining population density.

Some hunter-gatherer societies in Australia were studied, and the population density per 100 km² recorded for each. The data are in <http://www.utsc.utoronto.ca/~butler/c32/hg.txt>.

- (a) Read the data into R. Do you have the correct variables? How many hunter-gatherer societies in

Australia were studied? Explain briefly.

Solution: The data values are separated by (single) spaces, so `read_delim` is the thing:

```
url="http://www.utoronto.ca/~butler/c32/hg.txt"
societies=read_delim(url," ")

## Parsed with column specification:
## cols(
##   name = col_character(),
##   density = col_double()
## )
```

I like to put the URL in a variable first, because if I don't, the `read_delim` line can be rather long. But if you want to do it in one step, that's fine, as long as it's clear that you are doing the right thing.

Let's look at the data frame:

```
societies

## # A tibble: 13 x 2
##   name      density
##   <chr>      <dbl>
## 1 jeidji      17
## 2 kuku        50
## 3 mamu        45
## 4 ngatjan    59.8
## 5 undanbi    21.7
## 6 jinibarra   16
## 7 ualaria     9
## 8 barkindji  15.4
## 9 wongaibon   5.12
##10 jaralde    40
##11 tjapwurong  35
##12 tasmanians 13.4
##13 badjalang  13.4
```

I have the name of each society and its population density, as promised (so that is correct). There were 13 societies that were studied. For me, they were all displayed. For you, you'll probably see only the first ten, and you'll have to click Next to see the last three.

- (b) The question of interest is whether these Australian hunter-gatherer societies are like the rest of the world in terms of mean population density. State suitable null and alternative hypotheses. *Define any symbols you use:* that is, if you use a symbol, you also have to say what it means.

Solution: The mean for the world as a whole ("average", as stated earlier) is 7.38. Let μ denote the population mean for Australia (of which these societies are a sample). Then our hypotheses are:

$$H_0 : \mu = 7.38$$

and

$$H_a : \mu \neq 7.38.$$

There is no reason for a one-sided alternative here, since all we are interested in is whether Australia is different from the rest of the world.

Expect to lose a point if you use the symbol μ without saying what it means.

- (c) Test your hypotheses using a suitable test. What do you conclude, in the context of the data?

Solution: A t -test, since we are testing a mean:

```
t.test(societies$density,mu=7.38)

##
##  One Sample t-test
##
## data:  societies$density
## t = 3.8627, df = 12, p-value = 0.002257
## alternative hypothesis: true mean is not equal to 7.38
## 95 percent confidence interval:
##  15.59244 36.84449
## sample estimates:
## mean of x
## 26.21846
```

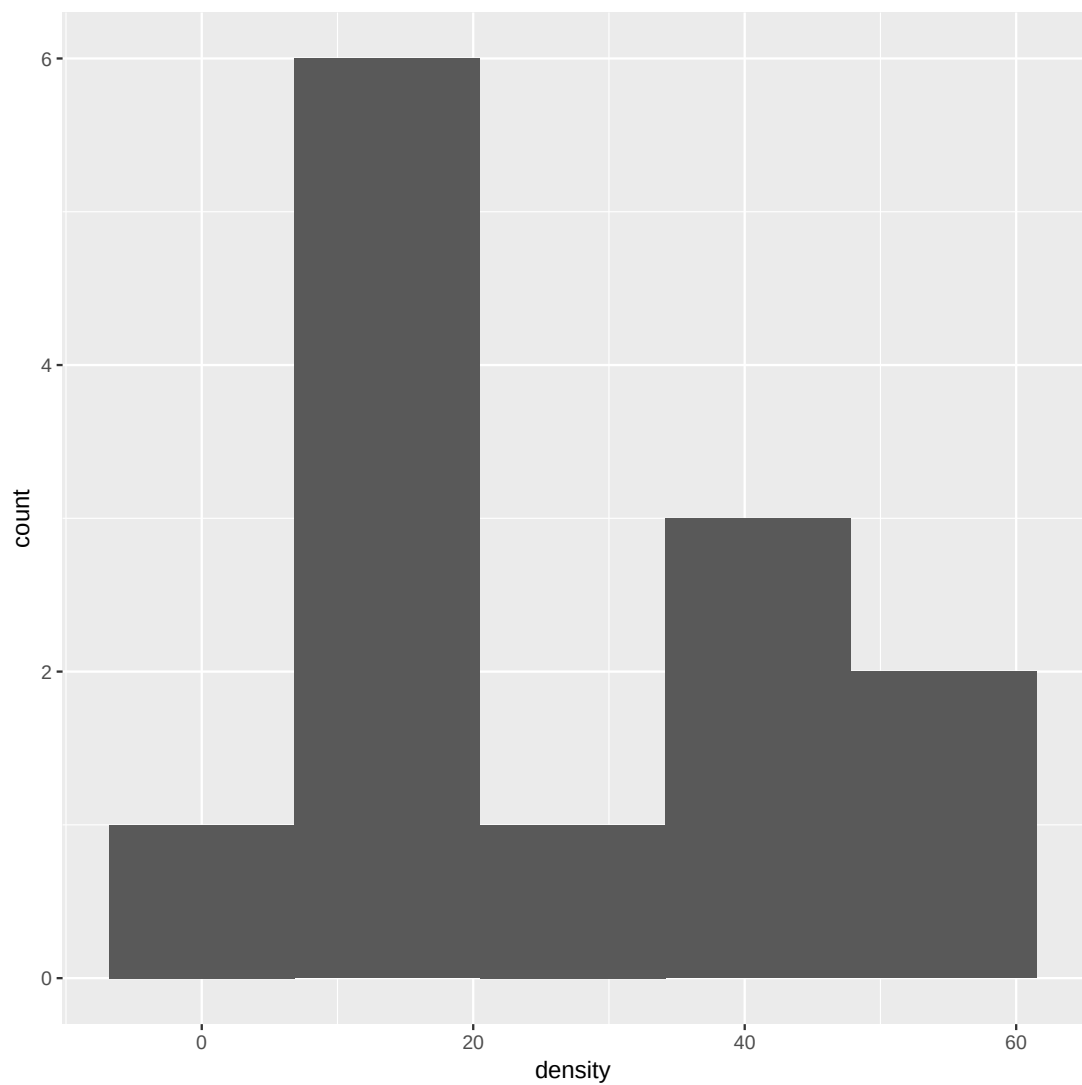
The P-value is 0.0023, less than the usual α of 0.05, so we *reject* the null hypothesis and conclude that the mean population density is not equal to 7.38. That is to say, Australia is different from the rest of the world in this sense.

As you know, “reject the null hypothesis” is only part of the answer, so gets only part of the marks.

- (d) Do you have any doubts about the validity of your test? Explain briefly, using a suitable graph to support your explanation.

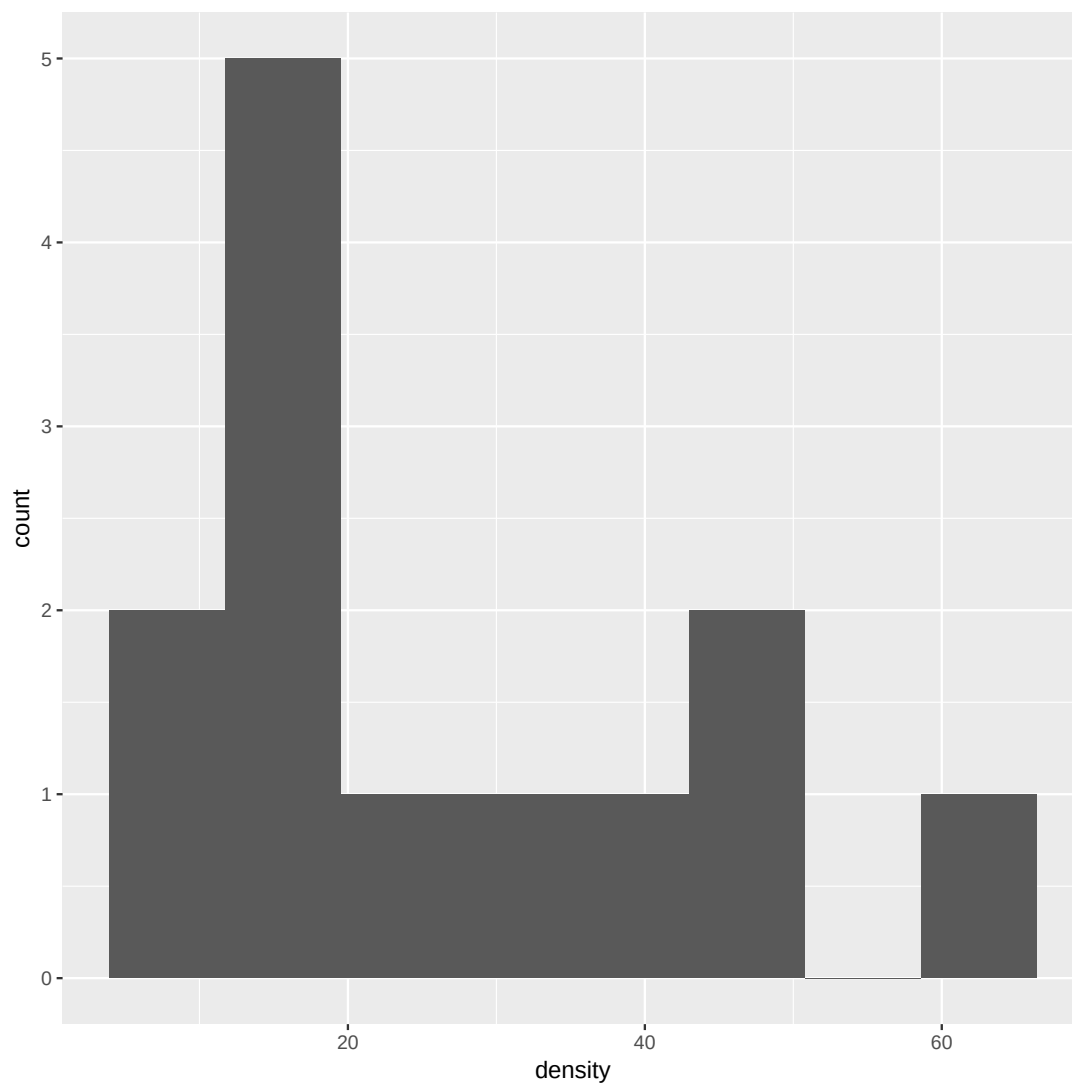
Solution: The assumption behind the t -test is that the data are approximately normal. We can assess that in several ways, but the simplest (which is perfectly acceptable at this point) is a histogram. You’ll need to pick a suitable number of bins. This one comes from Sturges’ rule:

```
ggplot(societies,aes(x=density))+geom_histogram(bins=5)
```



Your conclusion might depend on how many bins you chose for your histogram. Here's 8 bins (which is really too many with only 13 observations, but it actually shows the shape well):

```
ggplot(societies,aes(x=density))+geom_histogram(bins=8)
```



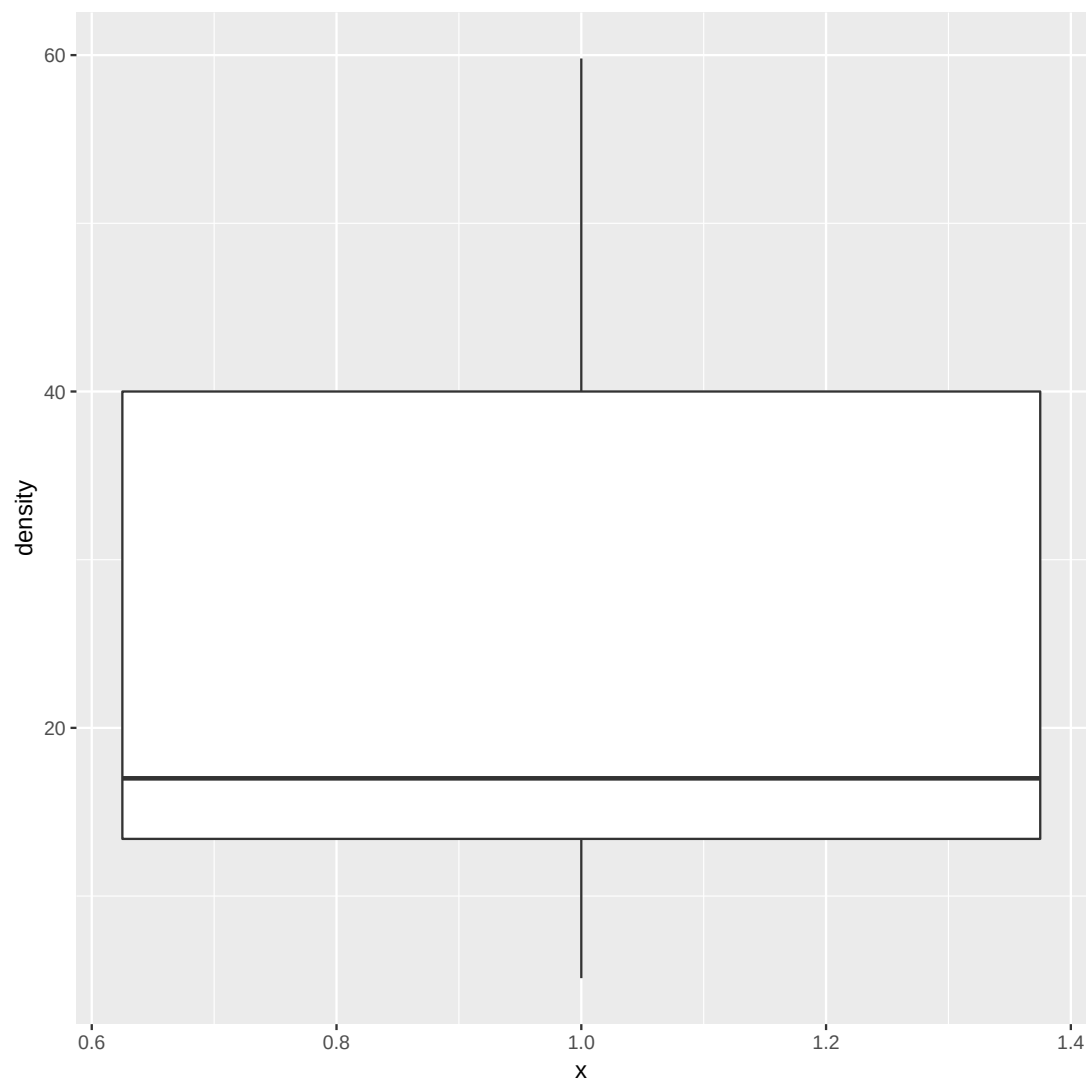
or you can get a number of bins from one of the built-in functions, such as:

```
mybins=nclass.FD(societies$density)
mybins
## [1] 3
```

This one is small. The interquartile range is large and n is small, so the binwidth will be large and therefore the number of bins will be small.

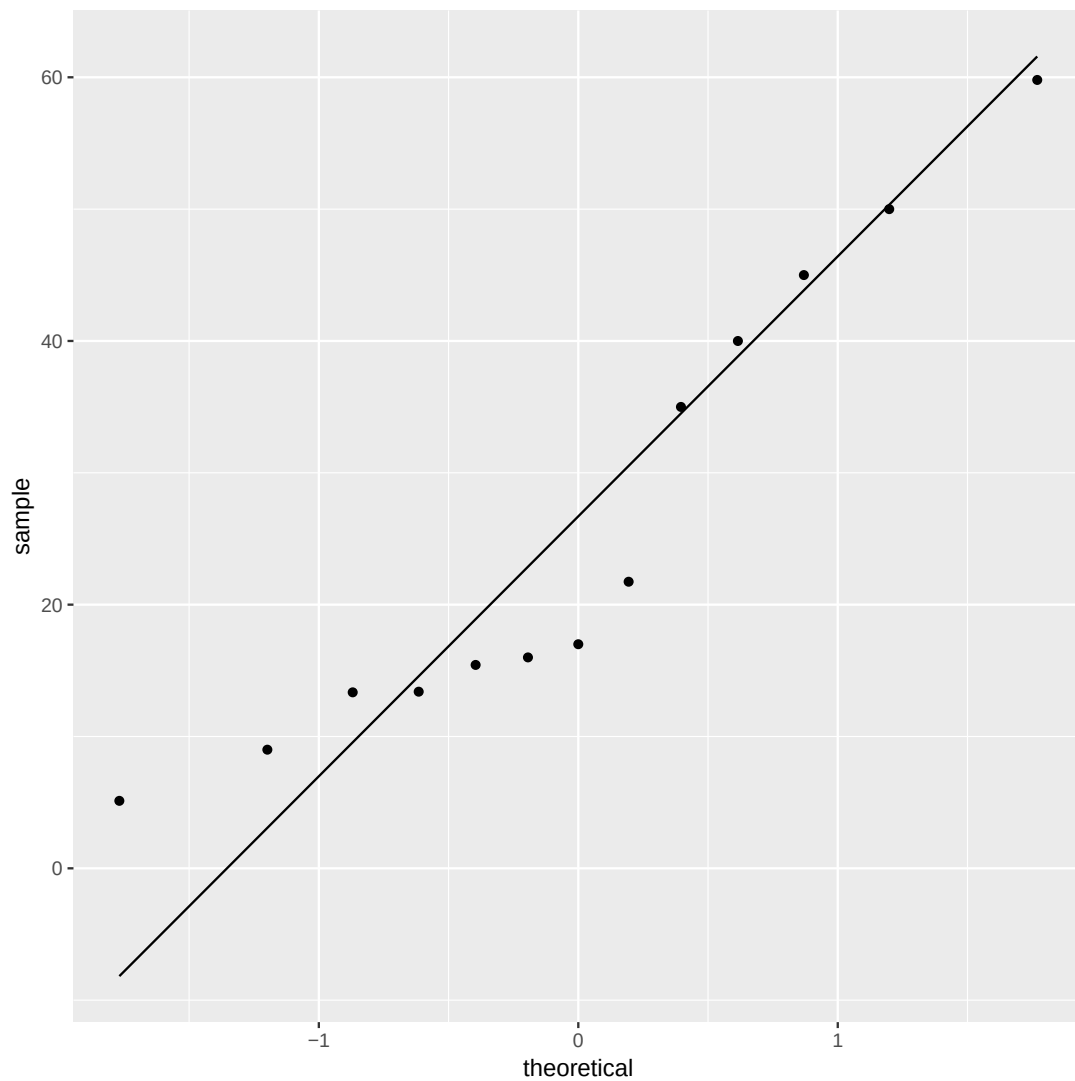
Other choices: a one-group boxplot:

```
ggplot(societies,aes(x=1,y=density))+geom_boxplot()
```



This isn't the best for assessing normality as such, but it will tell you about lack of symmetry and outliers, which are the most important threats to the t -test, so it's fine here. Or, a normal quantile plot:

```
ggplot(societies, aes(sample=density))+  
  stat_qq()+stat_qq_line()
```



This is actually the best way to assess normality, but I'm not expecting you to use this plot here, because we may not have gotten to it in class yet. (If you have read ahead and successfully use the plot, it's fine.)

After you have drawn your chosen plot (you need *one* plot), you need to say something about normality and thus whether you have any doubts about the validity of your t -test. This will depend on the graph you drew: if you think your graph is symmetric and outlier-free, you should have no doubts about your t -test; if you think it has something wrong with it, you should say what it is and express your doubts. My guess is that you will think this distribution is skewed to the right. Most of my plots are saying that.⁸

On the website where I got these data, they were using the data as an example for another test, precisely *because* they thought the distribution was right-skewed. Later on, we'll learn about the sign test for the median, which I think is actually a better test here.

Hand the next two questions in:

8. How is the speed limit on a stretch of road determined? The standard procedure in North America is to

wait until traffic is flowing smoothly on a piece of road, and then to record the speeds of a large number of cars. Of those speeds, the 85th percentile (the 0.85 quantile), rounded *down* to the next 5 miles per hour, is taken as the new speed limit.

The data in <http://www.utsc.utoronto.ca/~butler/c32/car-speeds.csv> are the speeds, in miles per hour, of cars on a certain stretch of road. Each car has a label. We are going (eventually) to determine what we think the new speed limit should be for this stretch of road.

- (a) (2 marks) Read the data into R and determine whether you have an appropriate number of observations of an appropriate number of variables.
 - (b) (2 marks) Draw a suitable plot of the speeds.
 - (c) (2 marks) How would you describe the *shape* of the distribution of the distribution of speeds, based on your graph? Explain briefly.
 - (d) (3 marks) Based on these data, what would be your recommendation for the new speed limit for this section of road? Show your thought process.
9. If you carry a backpack, it is recommended that you do not carry more than 10% of your body weight in it. A study was carried out at Cal Poly to see whether students were adhering to these recommendations. 100 students with backpacks, a mixture of males and females, were randomly sampled. Each student was weighed (without their backpack), then their backpack was weighed. Both weights were measured in pounds. The weight ratio (backpack weight divided by student's weight) was calculated. The student's gender was also recorded. The data are in <http://www.utsc.utoronto.ca/~butler/c32/Backpack.csv>.
- (a) (3 marks) Read the data into R and display the first few rows. What is the name of the column containing backpack weight divided by body weight?
 - (b) (2 marks) Make a suitable plot of the backpack weight - body weight ratios. You don't need to worry here about the gender of the students.
 - (c) (4 marks) Is there evidence that the mean backpack-to-body-weight ratio for all Cal Poly students (of whom these are a sample) is less than 0.10? Carry out a suitable test at $\alpha = 0.05$, and interpret the results in the context of the data.
 - (d) (2 marks) Obtain a 90% confidence interval for the mean weight ratio.
 - (e) (2 marks) Looking at the graph you drew, do you have any concerns about using a *t*-test here? Explain briefly.

Notes

¹these are mostly `int` or “integer”.

²Other possible variable types are `num` for (real, decimal) numbers such as birth weight, `chr` for text, and `Factor` (with the number of levels) for factors/categorical variables. We don't have any of the last two here. There is also `lg1` for “logical”, things that were actually recorded as TRUE or FALSE. We have some variables that are actually logical ones, but they are recorded as integer values.

³The backticks look different from each other for annoying technical reasons, but they're all backticks.

⁴When Tukey, a name we will see again, invented the boxplot in the 1950s, 500 observations would have been considered a big data set. He designed the boxplot to produce a sensible number of outliers for the typical size of data set of his day, but a boxplot of a large data set tends to have a lot of outliers that are probably not really outliers at all.

⁵I explain the missing values below.

⁶If there had been a weeks of gestation, we could have figured out whether it was premature or not, according to whether the weeks of gestation was less than 37.

⁷That is to say, the principal deviation from normality is not the “hole” on the histogram, the bar centred around 123 being too short, but that the bar centred just below 120 is too *tall*.

⁸The normal quantile plot is rather interesting: it says that the uppermost values are approximately normal, but the *smallest* eight or so values are too bunched up to be normal. That is, normality fails not because of the long tail on the right, but the bunching on the left. Still right-skewed, though.

⁹If the speeds had been recorded with, say, one decimal place, this would not have worked nearly so well.

¹⁰If you don’t define your μ first, any statements containing μ are *meaningless*.

¹¹By “with an equals” I mean either = itself, or something like $\geq$ with an equals in it.

¹²Which I didn’t put on this assignment, since I wasn’t sure whether we would have talked about it in class.

¹³What also works is to omit the . completely, but then the `with` doesn’t appear to have a data frame to work with, which looks rather odd.